

DriftSQL_{Implementation freeze update: 17 September 2026}

MASTER BLUEPRINT

An Enterprise Query Intelligence and Validation System

Integrated research foundation: Prompt 1 + Prompt 2 + Prompt 3

Literature-grounded | Implementation-ready | Two-member final-year project | Six-month scope

Primary research component: EEVA - Enterprise Evidence Validation Algorithm

Date: 10 August 2026

Research positioning

EEVA is central only in the validated narrow formulation: a query-conditioned, pre-generation validator over a normalized multi-source enterprise evidence state. Retrieval, schema linking, ambiguity handling, abstention/refusal, permission checks, SQL validation and self-correction are treated as existing/adopted supporting mechanisms. No global "first" claim is made.

Scope discipline

This blueprint preserves the validated research foundation and six-month scope. Expected results are hypotheses to be tested, not reported findings. Olist Brazilian E-Commerce is the approved primary database strategy, but its source, license, files and exact schema must be verified from the actual materials before dataset-specific claims are made. A small LiveSQLBench Base-Lite subset is reserved for secondary generalization evaluation only after the Olist implementation is stable and frozen.

Six pre-implementation specifications are now resolved. Appendix E is authoritative for B5/B6, Gold R(q), unsupported queries, clarification authority, freshness rules and team ownership. Core architecture unchanged.

Blueprint Contents

- 1. Project Title
- 2. One-Line Idea
- 3. Abstract
- 4. Project Overview
- 5. Background
- 6. Motivation
- 7. Problem Statement
- 8. Existing Systems
- 9. Existing System Limitations
- 10. Literature Survey
- 11. Literature Comparison
- 12. Research Gap
- 13. Research Objectives
- 14. Research Questions
- 15. Base Paper
- 16. Supporting Papers
- 17. Proposed System
- 18. Novelty
- 19. Research Contributions
- 20. Final Architecture
- 21. Architecture Explanation
- 22. Complete Data Flow
- 23. Query Intelligence Module
- 24. Enterprise Context Retrieval Module
- 25. EEVA Module
- 26. Evidence Decision Module
- 27. Explainable Join Reasoning
- 28. Text-to-SQL Generation
- 29. SQL Validation
- 30. Self-Correction
- 31. Controlled Execution
- 32. Explainable Result
- 33. Dataset
- 34. Enterprise Evidence Construction
- 35. Evidence Drift Scenario Generation
- 36. Algorithms
- 37. Pseudocode
- 38. Technology Stack
- 39. Baseline Models
- 40. Experimental Design
- 41. Ablation Study
- 42. Evaluation Metrics

- 43. Expected Results
- 44. Limitations
- 45. Six-Month Timeline
- 46. Team Member Responsibilities
- 47. Risks and Mitigation
- 48. Future Work
- 49. Research Paper Contribution
- 50. Final Viva Explanation
- Appendix A. 30-Paper Traceability
- Appendix B. DriftSQL Module Traceability
- Appendix C. Novelty Traceability and Targeted Cross-Check
- Appendix D. Final Consistency Check
- Appendix E. Implementation Freeze - Resolved Pre-Implementation Specifications

1. Project Title

DriftSQL: An Enterprise Query Intelligence and Validation System

Short project name

DriftSQL

2. One-Line Idea

One-line research idea

DriftSQL validates whether the enterprise evidence required by a natural-language query is sufficiently complete, current, consistent, recoverable and authorized before an LLM is allowed to generate SQL.

3. Abstract

Large language models have improved Text-to-SQL generation, but enterprise deployment remains vulnerable to a different failure source: the context supplied to the generator can itself be incomplete, stale, ambiguous, contradictory, or unauthorized. DriftSQL is a research-oriented enterprise Text-to-SQL system that inserts a pre-generation evidence-validation stage before SQL synthesis. Its central proposed component, EEVA (Enterprise Evidence Validation Algorithm), receives query-conditioned Required Evidence Obligations $R(q)$ and a normalized evidence bundle spanning schema, metadata, business glossary, documentation, relationship/value facts, and permission policies. EEVA applies non-compensatory hard constraints for critical schema, relationship, conflict, provenance, and authorization failures, while using recoverability logic for missing, stale, ambiguous, or resolvable conflicting evidence. It returns GENERATE, CLARIFY, or REJECT with explicit reason codes. If generation is allowed, DriftSQL performs explainable join reasoning, LLM-based SQL generation, a separate Stage-2 SQL-validation stage, at most two SQL-correction attempts, controlled read-only execution, and an explainable result. The primary implementation and detailed case study use the real anonymized multi-table Olist Brazilian E-Commerce database with a constructed enterprise evidence layer and controlled evidence-failure scenarios. A small LiveSQLBench Base-Lite subset is used only after Olist is stable/frozen for secondary cross-database generalization, with results reported separately. The key causal experiment compares identical B5 and B6 pipelines, differing only by the Stage-1 EEVA gate. DriftSQL proposes and investigates a query-conditioned enterprise evidence-validation mechanism for Text-to-SQL.

4. Project Overview

DriftSQL allows a non-SQL user to ask a business question in natural language and receive either a validated query result or a transparent explanation of why generation cannot safely proceed. The system is explicitly split into two validation stages. Stage 1 (EEVA) validates the prerequisite enterprise evidence before a SQL candidate exists. Stage 2 validates the generated SQL itself. This separation is essential to the research design and to the ablation study.

- Input: natural-language business question.
- Stage 1: query intelligence -> evidence retrieval/normalization -> EEVA -> GENERATE / CLARIFY / REJECT.
- Generation path: explainable join reasoning -> LLM Text-to-SQL.
- Stage 2: SQL validation -> bounded self-correction if needed -> controlled read-only execution.
- Output: SQL/result when safe, plus evidence provenance, join path, validation status, and clarification/rejection reasons.

5. Background

The selected literature shows a progression from generation-focused Text-to-SQL toward more modular reliability mechanisms. SC-Prompt [P1] decomposes SQL structure and content; FinSQL [P2] and CodeS [P3] improve domain/schema handling; modern agentic systems such as OpenSearch-SQL [P6], DeepEye-SQL [P8], SDE-SQL [P16], PV-SQL [P17], LitE-SQL [P18], and XiYan-SQL [P21] combine retrieval, reasoning, verification, execution feedback, or candidate selection. Separate work addresses ambiguity [P10], unanswerability [P11], schema generalization [P13], knowledge augmentation [P14], and retrieval quality [P19]. The Prompt 1 synthesis therefore established that DriftSQL must not position ordinary Text-to-SQL generation, RAG, abstention, or post-generation verification as new research by themselves.

6. Motivation

Academic benchmarks often provide relatively clean schema representations, while enterprise use introduces wide schemas, cryptic names, evolving metadata, business-specific terminology, documentation versions, access restrictions, and real-world join structures. FinSQL [P2] demonstrates difficulty with wide abbreviated financial schemas; the real-world database study [P23] shows major benchmark-to-enterprise performance degradation and gains from LLM-friendly context; Baek et al. [P14] shows the value and privacy implications of external knowledge; RAGSQL [P19] shows that retrieval quality itself must be measured; and schema-generalization work [P13] shows sensitivity to structural change. These findings motivate a system that treats retrieved context as evidence to be validated, rather than as automatically trustworthy prompt material.

Core motivation

A syntactically valid SQL query can still be semantically wrong if the generator was grounded in the wrong business definition, stale schema, contradictory documentation, missing relationship information, or unauthorized evidence.

7. Problem Statement

Given a natural-language query q , an enterprise evidence store E , and a relational database D , determine whether the evidence required to translate q into SQL is sufficiently reliable for generation. Reliability is query-specific: evidence that is irrelevant to one question may be mandatory for another. The system must distinguish evidence failures that are recoverable (retrieve/refresh/ask) from critical failures that require rejection, and it must do so before the SQL generator runs.

Formal research problem

How can an enterprise Text-to-SQL system define query-conditioned evidence obligations across heterogeneous enterprise sources, validate their completeness, freshness/validity, consistency, conflicts, authorization and recoverability, and use that state to control SQL generation without excessive false rejection?

8. Existing Systems

System family	Representative papers	What it solves
Generation / prompting	SC-Prompt [P1], SLENet [P20], tailored prompting [E26]	Improve SQL synthesis and structure-aware decoding.
Schema linking / schema retrieval	FinSQL [P2], CodeS [P3], schema re-appraisal [P12], LitE-SQL [P18], XiYan-SQL [P21]	Find relevant tables/columns and improve robustness.
Pre-generation abstention / answerability	RTS [P4], AmbiQT [P10], DTE [P11]	Avoid forced answers under uncertainty or ambiguity.
Evidence / knowledge retrieval	DIVER [P7], Baek et al. [P14], SDE-SQL [P16], RAGSQL [P19]	Acquire values, facts, examples, and external/domain knowledge.
Post-generation verification / correction	SQLDriller [P5], OpenSearch-SQL [P6], DeepEye-SQL [P8], PV-SQL [P17], LitE-SQL [P18]	Validate or repair candidate SQL using rules/execution.
Enterprise / real-world context	FinSQL [P2], DB-GPT [P22], real-world database study [P23], DEXA real-world RAG [E29]	Expose enterprise schema, metadata, privacy, and database-specific challenges.

9. Existing System Limitations

Observed limitation	Literature basis	DriftSQL response
Retrieved context is often treated as usable once relevant	RAGSQL / Baek et al. / DIVER optimize or acquire context but do not establish a unified cross-source trust state [P7,P14,P19].	Place EEVA after retrieval and before generation.
Pre-generation reliability is narrow	RTS validates schema-link/model uncertainty [P4]; DTE handles ambiguity/unanswerability [P11].	Validate a heterogeneous evidence object instead of one uncertainty source.
Business/docs/policy are fragmented	Enterprise semantics and privacy are recognized [P14,P22,P23], but not unified as one query-conditioned validation state in the reviewed corpus.	Normalize these evidence types with provenance/version/authority.
Post-generation repair is too late for bad	SQLDriller, DeepEye-SQL, PV-SQL, LitE-SQL	Keep Stage 2, but prevent avoidable unsafe

Observed limitation	Literature basis	DriftSQL response
prerequisites	operate once SQL/reasoning already exists [P5,P8,P17,P18].	generation in Stage 1.
Always-reject is not useful	Abstention work shows reliability/coverage trade-offs [P4].	Use CLARIFY for recoverable failures and measure FRR/recovery.

10. Literature Survey

10.1 Generation architectures and reasoning

SC-Prompt [P1] decomposes few-shot generation into structure and content stages; FinSQL [P2] combines schema retrieval, model adaptation and calibration for finance; CodeS [P3] develops open Text-to-SQL models; OpenSearch-SQL [P6] uses dynamic few-shot context, intermediate SQL-like reasoning and refinement; Reward-SQL [P9] and execution-feedback optimization [P15] improve reasoning using executable supervision; SLENet [P20] combines prompt search, LLM-enhanced embeddings and syntax-constrained decoding; XiYan-SQL [P21] uses multiple schema candidates and generators. These works demonstrate mature generation techniques but do not establish that the supplied enterprise context is trustworthy.

10.2 Schema linking, reasoning, and drift

FinSQL [P2], CodeS [P3], schema-linking re-appraisal [P12], LitE-SQL [P18] and XiYan-SQL [P21] show that schema selection is a major determinant of Text-to-SQL quality. RTS [P4] adds calibrated pre-generation abstention when schema linking is uncertain. Exploring Schema Generalizability [P13] demonstrates that structurally transformed schemas can sharply degrade models. DeepEye-SQL [P8] and PV-SQL [P17] add relationship reasoning or database probing. For DriftSQL, this literature supports schema obligations and freshness checks, but schema alone is only one evidence type.

10.3 RAG, knowledge, and evidence acquisition

Baek et al. [P14] constructs and retrieves reusable domain knowledge; RAGSQL [P19] explicitly evaluates context retrieval quality; DIVER [P7] dynamically gathers and verifies value/fact evidence; SDE-SQL [P16] uses exploratory SQL probes; PV-SQL [P17] probes the database before downstream verification. These papers strongly support evidence acquisition and CLARIFY recovery, while also exposing the distinction between retrieving relevant information and validating whether the resulting evidence set is sufficient, current, consistent and authorized.

10.4 Ambiguity, answerability, and abstention

AmbiQT [P10] shows that one natural-language question can legitimately correspond to multiple SQL interpretations. DTE [P11] detects and explains ambiguous or unanswerable questions. RTS [P4] provides calibrated abstention and feedback for uncertain schema linking. ARE-SQL [P25] tracks conversational semantics across turns. These works justify non-forced behavior and user clarification, but the trigger is usually linguistic/model/schema uncertainty rather than a cross-source enterprise evidence state.

10.5 SQL validation, execution feedback, and self-correction

SQLDriller [P5] uses execution consistency and equivalence/counterexamples; DeepEye-SQL [P8] uses checker chains and execution clustering; PV-SQL [P17] translates requirements into executable verification rules; LitE-SQL [P18] performs execution-guided self-correction; OpenSearch-SQL [P6], Reward-SQL [P9], SDE-SQL [P16], and execution-feedback optimization [P15] also use execution feedback. This area is established and therefore becomes DriftSQL Stage 2, not the EEVA novelty claim.

10.6 Enterprise and real-world deployment

FinSQL [P2], CodeS [P3], Baek et al. [P14], DB-GPT [P22], and the real-world database study [P23] motivate privacy, large schemas, business terminology and context engineering. The DEXA real-world RAG paper [E29] similarly constructs database-specific retrieval examples from schema/documentation. These works support the enterprise framing but do not eliminate the need to verify agreement among schema, metadata, documentation, glossary and policy evidence.

Literature synthesis

The reviewed research is rich but fragmented by validation target: schema confidence, ambiguity, retrieval, database probing, access control, SQL verification, and execution feedback are individually established. DriftSQL is defensible only if EEVA is implemented and evaluated as the integration over a query-conditioned multi-source evidence state.

11. Literature Comparison

ID	Paper / shorthand	Area	Method	Dataset	EEVA relationship	Relevance
P1	SC-Prompt	Few-shot generation	Two-stage structure/content prompting	Spider; CoSQL; GeoQuery	Generation only; no evidence trust gate	Medium
P2	FinSQL	Enterprise/schema retrieval	Cross-Encoder retrieval + PEFT + calibration	BULL	Partial schema/enterprise overlap	High
P3	CodeS	Open/private Text-to-SQL	SQL-centric pretraining + SFT + schema linking	Spider; BIRD; robustness sets	Indirect; model/schema layer	Medium
P4	Reliable Text-to-SQL with Adaptive Abstention	Reliability/abstention	BPP + conformal prediction + feedback	BIRD; Spider	Strong partial; closest pre-generation control	High
P5	SQLDriller	SQL validation	Execution consistency + equivalence/counterexamples	Spider; BIRD	Complementary Stage 2	High
P6	OpenSearch-SQL	Agentic generation/retrieval	Dynamic few-shot + consistency alignment + refinement	BIRD	Partial context/alignment overlap	High
P7	DIVER	Evidence reasoning	Interactive value linking + CoTF evidence reasoning	BIRD; Spider; Dr.Spider; Spider-DK	Strong partial; evidence acquisition/reasoning	High
P8	DeepEye-SQL	System reliability	Robust linking + N-version generation + checker chain	BIRD; Spider; Spider 2.0	Partial; mostly schema/SQL verification	High
P9	Reward-SQL	Reasoning/RL	CoCTE + process-supervised rewards	BIRD; Spider	Low direct overlap	Medium
P10	AmbiQT / LogicalBeam	Ambiguity	Ambiguity benchmark + diverse logical beam	AmbiQT/Spider-derived	Partial; supports CLARIFY	High
P11	DTE	Answerability	Detecting-Then-Explaining + counterfactual data	User study + counterfactual sets	Strong partial decision semantics	High
P12	Schema Linking Re-appraisal	Schema robustness	EMSL re-appraisal + grammar linking	Spider variants	Low-partial schema evidence	Medium
P13	Schema Generalizability	Schema drift/generalization	Schema transformations + adversarial evaluation	Spider/transformed suites	Partial freshness/drift motivation	High
P14	Baek et al.	Knowledge augmentation	KB construction + vector retrieval/refinement	BIRD; Spider; CSTINSIGHT	Partial evidence acquisition	High
P15	Execution Feedback / ExCoT-DPO	Generation optimization	Execution-grounded CoT + DPO	BIRD; Spider	Low; post-context reasoning	Medium
P16	SDE-SQL	Database exploration	Self-driven SQL probes	BIRD; Spider	Partial recoverability/acquisition	High
P17	PV-SQL	Probing + verification	DB probing + rule-based SQL verification	BIRD; Spider/Mini-Dev	Strong partial but split across stages	High
P18	LitE-SQL	Efficient retrieval/correction	Vector schema retriever + execution correction	BIRD; Spider 1.0	Partial retrieval; low EEVA decision	High
P19	RAGSQL	RAG retrieval quality	Dense retrieval evaluation	BIRD-derived corpus	Upstream complement: relevance != trust	High
P20	SLENet	Parsing/generation	Search prompt + LLM embedding + syntax decoder	Spider	Low direct overlap	Medium
P21	XiYan-SQL	Multi-generator/schema coverage	Multi-path schema filters + generators + selector	BIRD; Spider	Partial evidence coverage	High

ID	Paper / shorthand	Area	Method	Dataset	EEVA relationship	Relevance
P22	DB-GPT	LLM + database systems	DB-specific prompting/fine-tuning vision	TPC workloads; IMDB	Background enterprise context	Medium
P23	Real-World Databases	Enterprise context	Benchmark + LLM-friendly views/samples	IndDB; Mondial	Strong motivation; no evidence gate	High
P24	SV2-SQL	Unverifiable in corpus	Full source missing	Not verifiable	No evidence-based comparison possible	Low
P25	ARE-SQL	Conversational alignment	State tracking + relation-aware attention	CHASE	Low; clarification state only	Low-Medium

12. Research Gap

Validated gap

Within the 25-paper literature foundation plus the targeted 2025-2026 Text-to-SQL cross-check used in Prompt 2, no exact equivalent was verified for a query-conditioned pre-generation mechanism that: (1) represents required evidence across schema, metadata, business glossary, documentation, relationships/values and authorization; (2) evaluates cross-source completeness, freshness/validity, consistency, conflicts and recoverability; and (3) uses that evidence state to decide GENERATE, CLARIFY or REJECT before the SQL generator runs. Existing work solves important subsets, so the research contribution is the formalized integration and its measurable effect, not the individual checks.

This is a scoped research hypothesis, not a global novelty proof. Broader enterprise metadata governance, semantic-layer and data-catalog literature should still be searched before publication if the paper intends to use stronger novelty wording.

13. Research Objectives

1. Define a machine-readable, query-conditioned enterprise evidence model covering schema, metadata, business definitions, documentation, relationship/value facts and authorization evidence.
2. Design EEVA as a non-compensatory pre-generation decision mechanism that labels each query-evidence state as GENERATE, CLARIFY or REJECT and emits explicit reason codes.
3. Measure whether EEVA reduces false generation and unsafe/unsupported SQL under controlled evidence failures compared with the same DriftSQL pipeline without EEVA.
4. Evaluate detection performance for missing evidence, conflicts, stale evidence and permission violations using held-out evidence scenarios.
5. Measure whether the CLARIFY loop recovers from recoverable evidence failures without creating excessive false refusals.
6. Quantify downstream impact on execution accuracy, valid SQL rate, execution errors and correction workload while keeping SQL validation as separate Stage 2.

14. Research Questions

- RQ1: Does query-conditioned pre-generation enterprise evidence validation reduce false generation of incorrect, unsupported or unauthorized SQL compared with the same Text-to-SQL pipeline without EEVA?
- RQ2: How accurately can EEVA classify enterprise evidence states into GENERATE, CLARIFY and REJECT across clean, missing, stale, conflicting and unauthorized evidence scenarios?
- RQ3: How robust is EEVA to schema and metadata drift, and does evidence refresh after CLARIFY restore downstream Text-to-SQL execution accuracy?
- RQ4: Can EEVA detect cross-source semantic/documentation conflicts and distinguish recoverable conflicts from conditions that require rejection?
- RQ5: What reliability-utility trade-off does EEVA introduce, measured by false acceptance, false rejection, execution accuracy and latency?

15. Base Paper

15.1 Selected base paper: Reliable Text-to-SQL with Adaptive Abstention [P4]

Prompt 1 selected P4 as the single best methodological base paper because its central intervention occurs before SQL generation, explicitly treats reliability rather than only raw accuracy, and permits abstention/feedback rather than forced generation. It monitors uncertain schema-linking branching points with a Branching Point Predictor and conformal calibration, then continues or requests assistance. This is closer to EEVA's control role than evidence-construction-only or post-generation verification systems.

Aspect	Blueprint decision
Problem	Unsafe SQL caused by uncertain schema linking and lack of reliable abstention.

Aspect	Blueprint decision
Why it is closest	Pre-generation reliability gate controlling whether downstream generation should continue.
Limitation relevant to DriftSQL	Validation target is schema-link/model uncertainty, not heterogeneous enterprise evidence across glossary/docs/metadata/policy.
What DriftSQL adopts	Selective pre-generation control, recovery instead of forced generation, reliability/coverage trade-off thinking.
What DriftSQL changes	The validation object becomes a normalized query-specific evidence state, with explicit hard constraints, cross-source conflict logic and three outcomes.

15.2 Why DIVER is supporting rather than base

DIVER [P7] is the strongest evidence-reasoning support paper because it constructs and verifies database facts before generation. Its primary problem, however, is evidence acquisition and reasoning. DriftSQL's central problem begins after evidence has been retrieved/constructed: whether the whole enterprise evidence state is reliable enough to permit generation. For that reason P4 remains the base paper and DIVER becomes a direct supporting paper.

16. Supporting Papers

Class	Papers	Role in DriftSQL
A - Directly relevant	P7 DIVER; P11 DTE; P14 Baek et al.; P17 PV-SQL; P19 RAGSQL; P23 Real-World DB Study	Evidence reasoning/acquisition, clarify/reject behavior, knowledge retrieval, verification boundaries, enterprise context.
B - Strong supporting	P2 FinSQL; P5 SQLDriller; P6 OpenSearch-SQL; P8 DeepEye-SQL; P10 AmbiQT; P13 Schema Generalizability; P16 SDE-SQL; P18 LitE-SQL; P21 XiYan-SQL	Enterprise/schema retrieval, post-generation verification, ambiguity, drift, probing, efficient retrieval/correction, schema coverage.
C - Background/supporting	P1 SC-Prompt; P3 CodeS; P9 Reward-SQL; P12 Schema Re-appraisal; P15 execution-feedback optimization; P20 SLENet; P22 DB-GPT; P25 ARE-SQL	Generation, privacy/local models, schema robustness, reasoning, broader DB+LLM context, conversational alignment.
D - Unverifiable	P24 SV2-SQL	Full paper was not present in the uploaded corpus; do not use it for technical claims until supplied.

17. Proposed System

The proposed system is a single pipeline centered on EEVA. The generator is intentionally replaceable; the research variable is the presence and behavior of the evidence-validation gate, not the brand or scale of the LLM. Earlier experimental acronyms are simplified according to Prompt 2: LCHR is optional and omitted from the research architecture; AASE/SDARE/CARD/SFO are merged into ordinary query/retrieval/validation functions; HF is removed as a separate named module.

Design principle

Validate evidence first; generate SQL second. Validate generated SQL separately after it exists. Never let Stage 2 be used as a substitute for Stage 1.

18. Novelty

Element	Classification	Conservative interpretation
Multi-source enterprise evidence validation	COMBINED / POTENTIALLY NOVEL (scoped)	Existing ingredients are distributed across P4, P7, P14, P19, P23 and Prompt 2 access-control cross-checks; exact unified evidence state not verified.
Schema completeness	ADAPTED	Schema retrieval/linking/abstention already exist [P2,P4,P18,P21].
Metadata validity/freshness	ADAPTED	Metadata enrichment/context engineering already exist [P23 and Prompt 2 X4].
Business glossary consistency	POTENTIALLY NOVEL (scoped)	Domain knowledge exists [P14,P23]; explicit competing-definition gating not verified in scoped Text-to-SQL literature.
Documentation consistency	POTENTIALLY NOVEL (scoped)	Knowledge retrieval exists; explicit versioned document conflict gate not verified in scoped Text-to-SQL literature.
Permission validation	EXISTING / ADOPTED	Prompt 2 cross-check X1-X3 shows permission-aware refusal/enforcement is established.
Evidence conflict detection	ADAPTED / COMBINED	Consistency/ambiguity checks exist [P5,P6,P10,P11,P17]; EEVA applies them to heterogeneous evidence facts.
Coverage/recoverability	ADAPTED / COMBINED	Retrieval coverage, alternative schema candidates, abstention coverage and active probes exist [P4,P7,P16,P19,P21].
GENERATE/CLARIFY/REJECT	COMBINED	Abstain/clarify/refuse behaviors exist [P4,P11 and Prompt 2 cross-checks]; contribution is evidence-state-driven unification.
Pre-generation validation	EXISTING in narrower forms	RTS and Prompt 2 cross-check VET show pre-generation intervention is not itself new.

Allowed novelty statement

DriftSQL proposes and investigates a query-conditioned enterprise evidence-validation mechanism that jointly evaluates heterogeneous evidence obligations before SQL generation and experimentally isolates whether this gate reduces false and unsafe generation. The supporting mechanisms are established or adopted, and no global "first" claim is made.

19. Research Contributions

Contribution	What is contributed
C1 - Query-conditioned enterprise evidence model	A normalized evidence representation and Required Evidence Obligations R(q) that make completeness/coverage query-specific instead of global.
C2 - EEVA decision policy	A non-compensatory validation procedure combining hard constraints (authorization, critical schema/relationship absence, critical conflict, provenance failure) with recoverability logic to produce GENERATE, CLARIFY or REJECT.
C3 - Controlled enterprise evidence-failure benchmark	A reproducible evidence layer and controlled perturbation suite over the Olist primary database, covering clean, missing, stale, ambiguous, conflicting, unauthorized and multi-failure evidence scenarios.

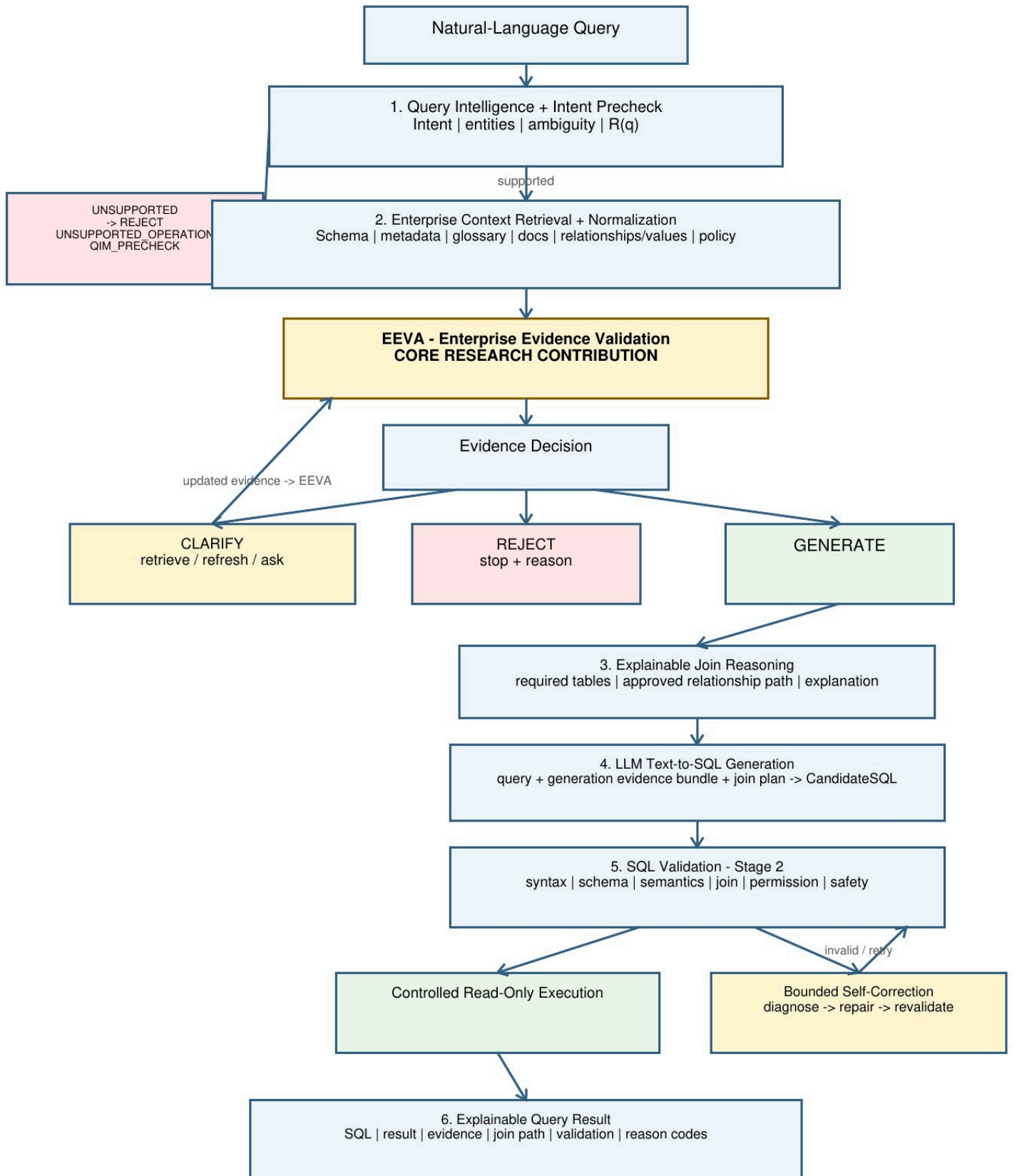
19. Research Contributions (continued)

Contribution	What is contributed
C4 - Causal evaluation of pre-generation validation	An identical-pipeline ablation (DriftSQL without EEVA vs with EEVA) using FAR/FRR, macro-F1, recovery, downstream execution accuracy and latency to measure the incremental value of EEVA.

Research contribution continuity

C4 completes the contribution table from page 12. The approved contribution remains the causal B5-vs-B6 evaluation of the Stage-1 EEVA gate; no new research claim is introduced on this continuation page.

20. Final Architecture



QIM precheck handles unsupported operations before retrieval/EEVA. Supported analytical queries follow the unchanged EEVA-centered two-stage architecture.

Figure 1. Final DriftSQL architecture. EEVA is Stage 1 evidence validation; SQL Validation is Stage 2 candidate-SQL validation.

21. Architecture Explanation

Module / transition	Meaning
Natural Language Query	User expresses a business question without SQL.
Query Intelligence	Extracts intent/entities/filters/metrics/ambiguity and creates Required Evidence Obligations R(q).
Enterprise Context Retrieval + Normalization	Retrieves candidate schema, metadata, glossary, documentation, relationship/value, and policy evidence and converts it into typed evidence records.
EEVA - Stage 1	Evaluates mandatory evidence, cross-source consistency, freshness, authorization, conflicts and recoverability before generation.
Evidence Decision	GENERATE continues; CLARIFY performs bounded retrieval/refresh/user clarification and returns to EEVA; REJECT stops with reason codes.
Explainable Join Reasoning	Constructs a valid table/relationship path only from evidence that passed EEVA.
LLM Text-to-SQL	Generates candidate SQL from the query, validated evidence bundle and join plan.
SQL Validation - Stage 2	Checks syntax, schema references, semantics, join consistency, permissions/safety, and controlled executability.
Self-Correction	Uses validation diagnostics to repair candidate SQL for a bounded number of attempts, then revalidates.
Controlled Execution	Executes only validated read-only SQL under DBMS permissions and resource limits.
Explainable Result	Returns result/SQL plus evidence provenance, EEVA decision, join path, SQL validation and correction details.

22. Complete Data Flow

1. Receive q and user/role context.
2. QIM normalizes q and creates QueryIntent plus Required Evidence Obligations R(q).
3. Retrieval obtains source-specific candidate evidence:
 - live schema catalog
 - curated/versioned metadata
 - approved business glossary
 - approved/versioned documentation
 - relationship/value facts via catalog and bounded probes
 - active role/policy evidence
4. Normalize each item into EvidenceItem records and attach provenance/version/authority/access labels.
5. EEVA evaluates every mandatory obligation and cross-source relationship.
- 6a. REJECT -> stop; emit hard-failure reason.
- 6b. CLARIFY -> produce recovery plan; retrieve/refresh/ask; merge changed evidence; return to EEVA (max 2 cycles).
- 6c. GENERATE -> freeze ValidatedEvidenceBundle for this query attempt.
7. Join reasoning builds a schema graph restricted to validated tables/relationships and selects an explainable join path.
8. LLM receives q + validated evidence + join plan + SQL-generation constraints; outputs CandidateSQL.
9. SQL validator parses and checks CandidateSQL, then performs controlled EXPLAIN/read-only validation as configured.
- 10a. Invalid -> bounded correction -> validation again.
- 10b. Valid -> controlled read-only execution.
11. Build final response with result, SQL, evidence summary, join path, Stage 1/Stage 2 status, latency and reason codes.

Data object	Minimum fields / purpose
QueryIntent	intent, entities, metrics, filters, temporal conditions, ambiguity flags, role/context
EvidenceObligation	requirement_type, target, mandatory, expected_source_types, constraint, diagnostic_weight
EvidenceItem	type, key, value, source, source_version, timestamp, scope, authority_level, access_label, provenance_id

Data object	Minimum fields / purpose
EEVAResult	decision, mandatory_coverage, evidence_states, hard_failures, conflicts, recovery_actions, provenance_summary, decision_reason
ValidatedEvidenceBundle	only evidence approved for SQL generation; immutable for one generation attempt
JoinPlan	required tables, relationship edges, chosen path, path explanation
SQLValidationResult	pass/fail, syntax/schema/semantic/join/policy/execution diagnostics
FinalResponse	decision/result, SQL when generated, explanation, evidence summary, join path, correction/validation status

FinalResponse	decision/result, SQL when generated, explanation, evidence summary, join path, correction/validation status
---------------	---

23. Query Intelligence Module

QIM is a supporting module that extracts structured intent and derives query-conditioned Required Evidence Obligations $R(q)$. The frozen implementation begins with a read-only scope precheck.

- Inputs: natural-language query, user role/context, and current date/time context when applicable.
- Intent precheck: INSERT/UPDATE/DELETE/DDDL/admin or other out-of-scope requests -> final REJECT with reason UNSUPPORTED_OPERATION and stage = QIM_PRECHECK; generator and executor are not called.
- Supported-query processing: normalization; entities/values; metrics/aggregation; filters/time/ranking; ambiguity; complexity; evidence-obligation derivation.
- Outputs: QueryIntent and $R(q)$, with mandatory/optional obligations and canonical targets where available.
- Evaluation only: one reviewed Gold $R(q)$ per base question is stored offline and is never accessible to runtime QIM, retrieval, EEVA or the generator.
- Linguistic ambiguity that prevents stable obligations creates a clarification requirement before evidence validation.

AASE decision

Aliases, abbreviations, synonyms and semantic mappings remain inside QIM/retrieval. Mappings must preserve provenance and normalize to canonical targets before Gold $R(q)$ scoring.

24. Enterprise Context Retrieval Module

Retrieval is deliberately hybrid. Live authoritative facts such as schema and active role policies should be looked up directly; semantic retrieval is used for glossary, documentation, metadata descriptions and relevant examples. RAGSQL [P19] motivates measuring retrieval independently; Baek et al. [P14] motivates knowledge retrieval; DIVER/SDE-SQL/PV-SQL [P7,P16,P17] motivate bounded evidence probing.

Evidence type	Primary source	Retrieval rule
Schema	PostgreSQL catalog / information_schema	Direct authoritative lookup; include version/hash.
Metadata	Curated metadata catalog + semantic index	Retrieve descriptions, aliases, units, date semantics; preserve version/provenance.
Business glossary	Approved versioned glossary	Semantic retrieval over business terms; return candidate definitions and authority/version.
Documentation	Approved versioned business-rule docs	Chunk + embed; top-k retrieval; preserve source ID/effective date.
Relationship/value facts	Catalog + bounded read-only probes	Validate FK paths, value formats and non-sensitive representative facts only when required.
Permission policy	PostgreSQL role/policy fixtures	Direct authoritative lookup; never rely only on vector retrieval for authorization.
Examples	Optional low-authority context	Useful for generation but never allowed to override mandatory authoritative evidence.

- Retrieval quality metrics: Recall@k and NDCG@k for semantic evidence.
- CLARIFY recovery: expand k, refresh direct sources, issue a safe probe, or ask the user depending on the failing state.
- Bound: at most two EEVA clarification cycles by default; repeated identical retrieval does not count as progress.

25. EEVA Module

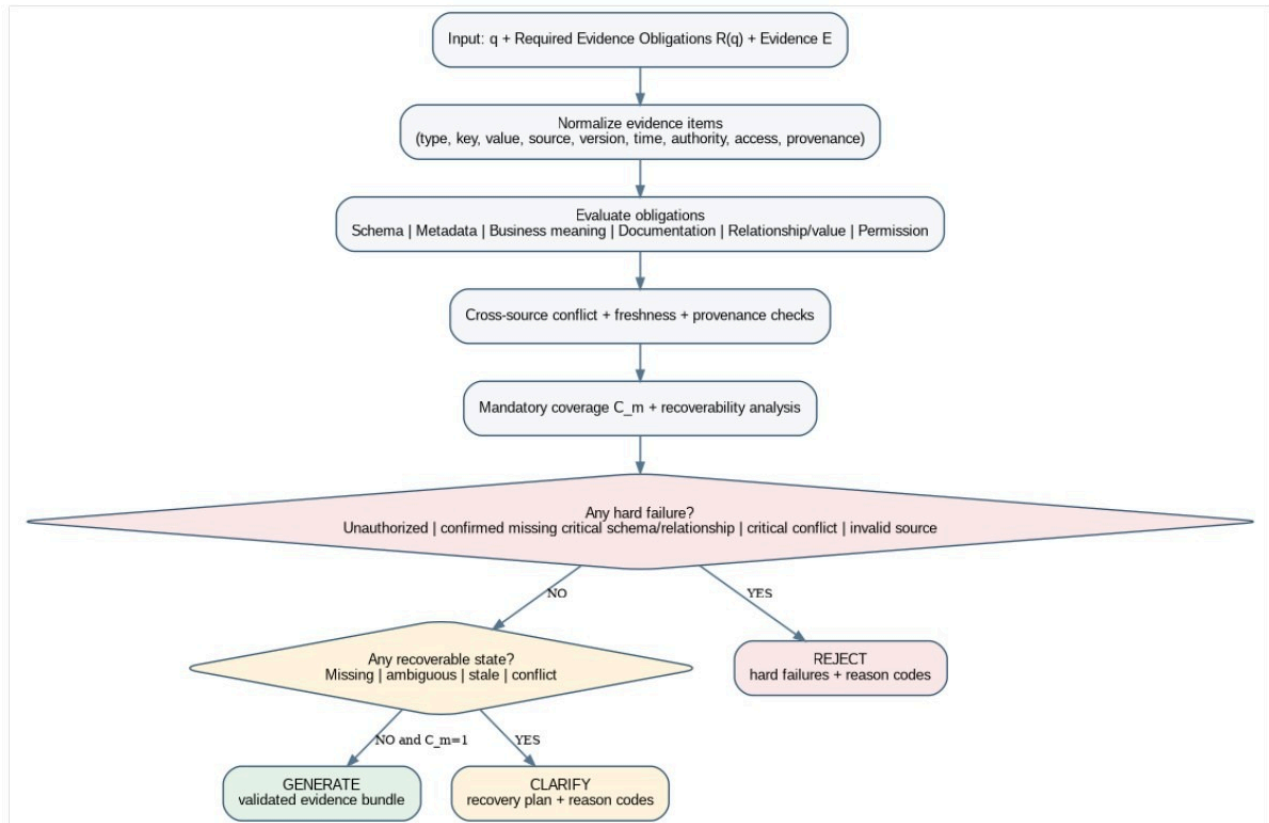


Figure 2. EEVA internal validation flow. Hard failures are non-compensatory; recoverable uncertainty routes to CLARIFY.

25.1 Input and evidence model

EEVA receives q , $R(q)$, the normalized evidence set E , and role/policy context. Each evidence item is typed rather than passed as unstructured prompt text:

```
e_i = {type, key, value, source, source_version, timestamp, scope, authority_level, access_label, provenance_id}
```

```
type in {SCHEMA, METADATA, GLOSSARY, DOCUMENTATION, RELATIONSHIP, VALUE, POLICY}
```

25.2 Mandatory versus optional evidence

- **Mandatory evidence:** facts without which the SQL meaning or authorization cannot be established for q (e.g., required table/column, join relationship, metric definition when ambiguous, active user authorization).
- **Optional evidence:** useful examples, extra descriptions or supporting value samples that improve interpretation but must not override hard failures.

25.3 Evidence states

State	Meaning
PASS	Required evidence exists, is current enough, consistent with authoritative sources, and authorized.
MISSING_RECOVERABLE	Absent from current retrieval but an approved retrieval/refresh/probe source remains.
AMBIGUOUS_RECOVERABLE	More than one plausible interpretation exists and can be resolved by user or authoritative evidence.
STALE_RECOVERABLE	Evidence is outdated but a live/current authoritative source is available.
CONFLICT_RECOVERABLE	Conflicting evidence exists but a live/current authoritative source is available.

State	Meaning
CONFLICT_CRITICAL	Authoritative sources remain contradictory after permitted recovery.
UNAUTHORIZED	Required information or operation is denied by active policy.
INVALID	Evidence fails structure/integrity/provenance checks and cannot safely support the query.

25.4 Hard constraints

- Authorization denial for mandatory data/operation -> REJECT.
- Mandatory table/column confirmed absent in authoritative current schema -> REJECT.
- Required relationship/join path unavailable with no supported alternative -> REJECT.
- Unresolved critical conflict between authoritative evidence sources -> REJECT.
- Critical evidence source fails integrity/provenance validation and no replacement exists -> REJECT.

25.5 Soft / recoverable constraints

- Missing metadata needed to interpret an existing cryptic column -> CLARIFY via metadata retrieval/profiling.
- Stale cached schema/metadata with live catalog available -> CLARIFY via refresh.
- Ambiguous business term with multiple approved definitions -> CLARIFY user or retrieve current rule.
- Document disagreement that can be resolved by version/authority -> CLARIFY and re-evaluate.
- Low retrieval coverage while approved sources remain -> CLARIFY and expand retrieval.

25.6 Formal coverage quantities

Let $R_m(q)$ be the mandatory obligations and E the retrieved evidence.

$C_m(q, E) = (\# \text{ mandatory obligations in PASS}) / (\# \text{ mandatory obligations})$

For optional obligations only:

$C_o(q, E) = \sum_j w_j * I[r_j \text{ satisfied}] / \sum_j w_j$

Non-compensation rule: no value of C_o may override a hard failure in permission, mandatory schema/relationship, or critical conflict.

Coverage is diagnostic, not a single global trust score. If a soft threshold is used for optional evidence, calibrate it on the

25.7 Source authority and frozen currentness policy

Evidence type	Authoritative current state	When stale/recoverable
Schema / relationships	Active PostgreSQL catalog + approved current relationship snapshot	Cached hash/version differs and live refresh exists.
Authorization	Active PostgreSQL role/policy configuration	Cached policy differs; denial itself is a hard failure.
Business glossary	Latest approved definition effective for the query	Older/superseded approved definition retrieved.
Documentation	Latest applicable approved version by authority/effective date	Older version retrieved while a current approved source exists.
Metadata	Current approved metadata snapshot linked to active schema	Version/hash mismatch and approved refresh is available.

26. Evidence Decision Module

Decision	Condition	Action
GENERATE	No hard failure; every mandatory obligation PASS; $C_m=1.0$.	Freeze a ValidatedEvidenceBundle and continue to join reasoning.

Decision	Condition	Action
REJECT	Any unauthorized/critical missing/critical conflict/critical invalid condition, or recovery bound exhausted with unresolved mandatory failure.	Stop before SQL generation and return transparent reason codes.

Critical clarification authority rule

After CLARIFY, updated QueryIntent/R(q) and evidence MUST return through retrieval/normalization and EEVA. A user may resolve intent, filters or select among approved interpretations, but cannot override schema, relationships, DB permissions/policy, provenance or authoritative enterprise definitions. Max 2 cycles; no progress/exhaustion -> REJECT with RECOVERY_EXHAUSTED.

27. Explainable Join Reasoning

This is a supporting mechanism grounded in schema/join reasoning work [P8,P17,P23]. NetworkX may be used as an implementation library, but graph reasoning is not a separate research contribution.

- Build a graph G where tables are nodes and validated PK/FK relationships are edges.
- Seed required table nodes from R(q) and validated schema evidence.
- Enumerate feasible join paths; reject edges not present in validated relationship evidence.
- Rank simple paths by path length, relationship authority, and required-table coverage; avoid arbitrary joins.
- Output a JoinPlan: tables, edge keys, join order, and a short explanation of why each edge is necessary.
- Failure: no validated relationship path -> this should normally have been caught by EEVA as a critical/recoverable relationship obligation; do not let the LLM invent a join.

28. Text-to-SQL Generation

The LLM is the generation engine, not the contribution. Freeze one primary instruction-following model for the main experiment so architecture, not model variation, determines the comparison. The prompt contains only the user query, ValidatedEvidenceBundle, JoinPlan, allowed SQL operation constraints, and minimal examples if used.

- Generation output: one CandidateSQL plus optional structured explanation fields; do not require hidden chain-of-thought.
- Keep deterministic decoding settings where feasible for reproducibility.
- Use the same model/prompt template in DriftSQL WITH and WITHOUT EEVA except for the presence of the EEVA gate/validated bundle.
- Do not allow the generator to access evidence that EEVA marked unauthorized or invalid.

29. SQL Validation

SQL Validation is Stage 2 and must remain conceptually separate from EEVA. It operates after CandidateSQL exists, drawing on established execution/checker literature [P5,P8,P17,P18].

Check	Implementation
Syntax	Parse SQL; reject parse errors or unsupported statements.
Statement safety	Allow read-only SELECT/CTE patterns only in project v1; reject DDL/DML.
Schema	Verify referenced tables/columns exist in the current authoritative catalog.
Join	Verify join keys/relationships agree with the validated JoinPlan or approved alternatives.
Semantic constraints	Check required filters/aggregation/time/ranking requirements derived from QueryIntent.
Permission/safety	Defense in depth: re-check active role access to referenced objects; DBMS remains authoritative.
Controlled execution preflight	Use parsing and/or PostgreSQL EXPLAIN in a read-only transaction with timeout before actual result execution.

30. Self-Correction

Self-correction is established supporting functionality [P6,P8,P17,P18] and should be bounded. It receives CandidateSQL, SQLValidationResult diagnostics, the original query, validated evidence and JoinPlan. It repairs only SQL-level failures; it must not silently bypass an EEVA decision or introduce new unauthorized evidence.

- Maximum correction attempts: 2 in the primary implementation.
- After each correction, run the full SQL-validation stage again.
- If the same failure repeats or the validator detects an underlying evidence mismatch, terminate and report failure rather than looping indefinitely.
- Record correction attempt count and success for the SQL Correction Success Rate metric.

31. Controlled Execution

Execution is a supporting safety mechanism and the DBMS is the final enforcement boundary. Recommended project controls:

- Dedicated read-only PostgreSQL role for the application.
- Explicit statement timeout and row/result-size limit.
- SELECT/CTE allow-list; no INSERT/UPDATE/DELETE/DDL in project v1.
- Transaction-scoped execution; rollback/no persistent mutation.
- Optional EXPLAIN preflight before result query.
- Log SQL hash, role, latency, validation result and execution status; avoid logging sensitive row values unnecessarily.

32. Explainable Result

The final response should make the two validation stages inspectable without exposing hidden model reasoning. The user-facing explanation is a structured audit summary, not an unrestricted chain-of-thought dump.

Case	Displayed / logged information
If GENERATE succeeds	Natural-language answer/result, generated SQL, tables/columns, join path, evidence source summary, EEVA status, SQL validation status, correction count, latency.
If CLARIFY requires user input	One focused clarification question plus the exact unresolved business/evidence reason; do not generate SQL yet.
If REJECT	Decision reason code(s), missing/conflicting/unauthorized requirement, and safe next action if one exists.
Research logging	Store oracle scenario ID, EEVA state, decision, reason codes, retrieval cycles, generation/validation/execution outcomes for experiment analysis.

33. Dataset

33.1 Primary dataset

Olist Brazilian E-Commerce is the primary database for PostgreSQL implementation, DriftSQL development, Streamlit/live demonstration, enterprise evidence-layer construction, the main detailed case study, and primary experimental evaluation. Treat it as a real anonymized multi-table e-commerce database. Verify the actual source, license, files, schema, keys and relationships before making dataset-specific claims; do not invent exact table names or columns. Keep the normal Olist transactional data unchanged. Controlled faults belong in evidence artifacts or controlled schema-drift copies.

33.2 Evaluation query and evidence-case plan

- Early demo: 20 base questions and approximately 100 evidence variants.
- Minimum final: 60-80 carefully reviewed base questions and approximately 300-400 evidence cases.
- Preferred final: approximately 100 base questions and approximately 500 evidence cases.
- Stretch: up to 120 base questions only if annotation quality remains high; quantity never overrides correctness.
- Keep every variant derived from the same base question in the same development/test partition to prevent leakage.
- Gold/reference SQL, expected results, evidence states and reason codes are offline-evaluation oracles only and are never supplied to runtime as hidden hints.

Dataset and secondary-evaluation discipline

Profile the actual Olist files and validate real keys/relationships plus a schema snapshot/hash before dataset-specific claims. After the Olist implementation is stable/frozen, run a small LiveSQLBench Base-Lite subset through the same DriftSQL codebase using database_id and a generic database adapter. Report Olist and LiveSQLBench results separately; never calculate one combined overall accuracy.

34. Enterprise Evidence Construction

Artifact	Contents	Construction
Schema snapshot	Live tables, columns, data types, PK/FK, schema version/hash.	Generate from PostgreSQL catalog; authoritative for structure.
Metadata catalog	Human-readable table/column descriptions, units, aliases, time semantics.	Curate/version; optionally supplement from profiling; attach provenance.
Business glossary	Approved e-commerce terms/metrics derived only from verified Olist fields and project definitions; no invented dataset semantics.	Create approved, versioned definitions linked to verified entities/metrics.
Documentation	Short business rules, formulas, relationship notes, effective dates.	Create versioned documents with source ID and authority.
Actual data / value facts	Allowed formats and representative non-sensitive samples/aggregates.	Use bounded read-only probes only when an obligation needs value evidence.
Relationship facts	PK/FK graph and any documented semantic relationship requirements.	From catalog; normalize to graph edges with provenance/version.
Permission policies	Role -> allowed tables/columns/operations.	Implement as PostgreSQL role/policy fixtures; authoritative security evidence.
Gold evidence obligations	Minimum facts needed for each evaluation query.	Manually annotated oracle used for EEVA evaluation, not fed as hidden hints.

35. Evidence Drift Scenario Generation

Scenario	Original state	Modified / generation method	Expected EEVA	Expected SQL behavior	Evaluation
1. Missing columns	Canonical evidence contains required column.	Remove required column from cached/retrieved schema evidence. For a critical-absence variant, remove/rename it in authoritative drift copy too.	CLARIFY if live catalog can recover; REJECT if authoritative schema confirms absence.	No SQL until EEVA re-evaluation; after recovery GENERATE is allowed.	Decision macro-F1, FAR, recovery rate, EX after recovery.
2. Renamed columns	Column has stable canonical name and metadata alias.	Rename physical column in drifted copy; leave stale cache/metadata referring to old name, with recoverable mapping available.	CLARIFY -> refresh/mapping -> GENERATE if resolved.	Without EEVA stale SQL may reference old name; with EEVA generation waits.	Drift detection, FAR, recovery EX.
3. Outdated documentation	Current versioned rule matches glossary/schema.	Keep an older calculation-rule document active in retrieval while a newer approved version exists.	CLARIFY -> fetch/current-version resolution.	Generate only after current rule is selected and consistent.	Freshness-state accuracy, FAR, recovery.
4. Documentation conflict	Approved sources agree.	Create two same-authority documents with contradictory formula/relationship statements.	CLARIFY if precedence/version can resolve; otherwise REJECT.	No generation under unresolved critical conflict.	Conflict P/R/F1, FAR.
5. Ambiguous business definition	Metric has one approved definition for context.	Define a term such as revenue with two plausible approved meanings for same domain/query.	User CLARIFY unless query context uniquely selects one.	Generate only after business meaning is resolved.	Clarify recall, FAR, post-clarification EX.
6. Missing metadata	Cryptic schema field has required description/unit/alias.	Remove metadata needed to interpret it while physical schema remains.	CLARIFY via metadata retrieval/profiling if semantics are required.	No blind semantic guess; generate after recovery.	Metadata-state accuracy, recovery rate, EX.
7. Changed relationships	Canonical FK/join relationship is current and indexed.	Alter FK/join mapping in drifted DB copy while cached relationship evidence remains stale.	CLARIFY -> refresh; REJECT if no valid path remains.	Join reasoning must use refreshed validated edge only.	Relationship detection, join accuracy, EX.
8. Permission restriction	Role is authorized for query objects.	Apply role policy denying required table/column/operation.	REJECT immediately as hard constraint.	Zero SQL generation for denied request; DBMS still enforces policy.	Permission detection, false permit, unsafe-generation rate.
9. Schema drift	Canonical schema/index/docs are synchronized.	Apply broader structural change: split/merge table, add/remove key, or change datatype/relationship, with stale retrieval index.	CLARIFY refresh if recoverable; REJECT if mandatory structure no longer supports query.	No generation from stale structural assumptions.	FAR, drift detection, recovery EX, latency.
10. Multiple evidence conflicts	All evidence dimensions consistent.	Combine 2-3 failures, e.g., stale schema + glossary conflict + restricted column.	Highest-severity unresolved hard state dominates; good optional evidence cannot compensate.	REJECT on hard failure or CLARIFY on recoverable-only set.	Macro-F1, FAR, reason-code accuracy.

36. Algorithms

Algorithm	Purpose	Research status
A. Query Intelligence	Parse q into QueryIntent and R(q).	Supporting; P6/P10/P11/P25.
B. Enterprise Evidence Retrieval	Retrieve source-specific evidence and normalize records.	Supporting; P7/P14/P16/P19/P23.
C. EEVA	Evaluate obligations, hard constraints, conflicts, coverage, freshness, authorization, recoverability.	CORE research algorithm.
D. Evidence Decision	Map EEVA states to GENERATE / CLARIFY / REJECT and recovery actions.	CORE decision policy, but decision labels themselves are combined/adapted.
E. Explainable Join Reasoning	Build validated schema graph and choose required join path.	Supporting; P8/P17/P23.
F. Text-to-SQL Generation	Generate SQL from q + validated evidence + JoinPlan.	Supporting; generation literature.
G. SQL Validation	Parse/check candidate SQL and controlled preflight.	Supporting/established; P5/P8/P17/P18.
H. Self-Correction	Repair SQL using validator diagnostics with bounded attempts.	Supporting/established; P6/P8/P17/P18.
I. Controlled Execution	Read-only, resource-bounded result execution.	Supporting safety mechanism.

37. Pseudocode

37.A Query Intelligence

```

INPUT: q, user_context
PROCESS:
  1. normalize_text(q)
  2. extract intent, entities, values, filters, metrics, time constraints
  3. detect linguistic ambiguity / answerability flags
  4. estimate query complexity and candidate domain concepts
  5. derive Required Evidence Obligations R(q):
    - schema/table/column obligations
    - relationship obligations
    - business-definition obligations
    - metadata/date/unit obligations
    - policy obligations
  6. mark each obligation mandatory or optional
OUTPUT: QueryIntent, R(q)
FAILURE CONDITIONS:
  - query cannot be parsed into a stable intent
  - unresolved user ambiguity -> produce user-clarification obligation

```

37.B Enterprise Evidence Retrieval

```

INPUT: QueryIntent, R(q), source_registry, user_context
PROCESS:
  1. fetch authoritative live schema facts for required structural targets
  2. retrieve top-k metadata/glossary/document chunks semantically
  3. fetch active permission/policy facts directly
  4. if required, perform bounded read-only value/relationship probes
  5. attach source, version, timestamp, authority, access label, provenance
  6. normalize all items to EvidenceItem records
  7. deduplicate by (type,key,source_version) without hiding conflicts
OUTPUT: normalized evidence set E + retrieval diagnostics
FAILURE CONDITIONS:
  - source unavailable -> mark recoverability, do not fabricate evidence
  - unauthorized source access -> return policy failure
  - retrieval yields no candidate for mandatory obligation -> preserve as missing

```

37.C EEVA - Enterprise Evidence Validation Algorithm

```

2. states <- empty map
3. reasons <- []
4. recovery <- []

5. FOR each obligation r in R(q):
  candidates <- evidence_matching(r, N)
  IF candidates is empty:
    IF permitted_recovery_source_exists(r):
      states[r] <- MISSING_RECOVERABLE
      recovery += recovery_action(r)
    ELSE IF r.mandatory:
      states[r] <- MISSING_CRITICAL
      reasons += MISSING_CRITICAL(r)
    ELSE:
      states[r] <- OPTIONAL_MISSING
      CONTINUE

  validate provenance/integrity of candidates
  validate current version/freshness policy
  evaluate source authority and precedence
  evaluate semantic agreement across candidates
  evaluate authorization for required target/operation

  IF authorization denied for mandatory r:
    states[r] <- UNAUTHORIZED
    reasons += UNAUTHORIZED(r)
  ELSE IF mandatory current schema confirms target absent:
    states[r] <- MISSING_CRITICAL
  ELSE IF required relationship has no valid approved path:
    states[r] <- MISSING_CRITICAL
  ELSE IF authoritative candidates remain contradictory and no recovery exists:
    states[r] <- CONFLICT_CRITICAL
  ELSE IF provenance/integrity invalid and no replacement exists:
    states[r] <- INVALID
  ELSE IF evidence is stale and a current source exists:
    states[r] <- STALE_RECOVERABLE
    recovery += REFRESH(r)
  ELSE IF multiple plausible business meanings remain:
    states[r] <- AMBIGUOUS_RECOVERABLE
    recovery += ASK_USER_OR_FETCH_AUTHORITY(r)
  ELSE IF sources conflict but precedence/version can resolve:
    states[r] <- CONFLICT_RECOVERABLE
    recovery += RESOLVE_CONFLICT(r)
  ELSE:
    states[r] <- PASS

6. C_m <- count(mandatory r with PASS) / count(mandatory r)
7. conflicts <- cross_source_conflict_scan(N, R(q))
8. hard_failures <- states in {UNAUTHORIZED, MISSING_CRITICAL, CONFLICT_CRITICAL, INVALID}

9. IF hard_failures not empty:
  RETURN EEVarResult(REJECT, C_m, states, hard_failures, conflicts, reasons)

10. recoverable <- states in {MISSING_RECOVERABLE, AMBIGUOUS_RECOVERABLE,
  STALE_RECOVERABLE, CONFLICT_RECOVERABLE}

11. IF recoverable not empty:
  RETURN EEVarResult(CLARIFY, C_m, states, [], conflicts, recovery_actions=recovery)

12. IF C_m == 1.0 AND all mandatory states == PASS:
  bundle <- freeze_validated_evidence(N, R(q))
  RETURN EEVarResult(GENERATE, 1.0, states, validated_bundle=bundle)

13. IF permitted_recovery_source_remains():
  RETURN CLARIFY
14. ELSE:
  RETURN REJECT

OUTPUT: decision + states + mandatory coverage + reason codes + recovery plan or validated bundle
FAILURE CONDITIONS:
- any hard constraint -> REJECT
- recoverable uncertainty -> CLARIFY
- recovery bound exhausted without mandatory PASS -> REJECT
NON-COMPENSATION:
optional evidence or high relevance can never override authorization denial, critical absence,
missing mandatory relationship, or critical authoritative conflict.

```

37.D Evidence Decision

```

IF decision == REJECT: stop; return reason codes
IF decision == CLARIFY:
  IF cycles >= 2: convert unresolved mandatory failure to REJECT
  ELSE execute only listed recovery actions:
    - refresh authoritative source
    - expand approved retrieval
    - bounded safe DB probe
    - ask one focused user clarification
  require evidence change/progress
  return updated E to EEVA
OUTPUT: continue / loop / stop
FAILURE CONDITIONS: no progress, recovery source unavailable, bound exhausted

```

37.E Explainable Join Reasoning

```

INPUT: QueryIntent, ValidatedEvidenceBundle
PROCESS:
  build graph from validated tables and relationship edges
  identify required table nodes
  enumerate feasible simple paths connecting required nodes
  remove paths using unvalidated/unauthorized edges
  rank by minimal valid path + authority + obligation coverage
  select path and generate edge-by-edge explanation
OUTPUT: JoinPlan
FAILURE CONDITIONS: no validated path -> stop/report relationship failure; do not invent join

```

37.F Text-to-SQL Generation

```

INPUT: q, QueryIntent, ValidatedEvidenceBundle, JoinPlan, SQL constraints
PROCESS:
  build compact prompt using only validated evidence
  invoke frozen primary LLM
  request SQL-only/structured output
  strip formatting wrapper; preserve candidate for validation
OUTPUT: CandidatesSQL
FAILURE CONDITIONS: empty/non-SQL response, model/tool failure -> report generation failure

```

37.G SQL Validation

```

INPUT: CandidatesSQL, QueryIntent, JoinPlan, DB catalog, active role
PROCESS:
  parse SQL
  enforce SELECT/CTE-only policy
  validate table/column existence
  validate joins against JoinPlan / approved relationships
  validate required filters, grouping, ranking, time constraints where checkable
  re-check referenced-object permissions
  run bounded EXPLAIN/read-only preflight as configured
OUTPUT: PASS or FAIL + structured diagnostics
FAILURE CONDITIONS: parse error, unsafe statement, invalid refs, join mismatch, policy denial, preflight error

```

37.H Self-Correction

```

INPUT: CandidatesSQL, validation diagnostics, q, validated evidence, JoinPlan
PROCESS:
  FOR attempt in 1..2:
    ask generator to repair only the diagnosed SQL-level issue
    re-run SQL Validation
    IF PASS: return corrected SQL
    IF identical failure/no progress: break
OUTPUT: corrected SQL or correction failure
FAILURE CONDITIONS: correction bound reached; repair needs evidence not in validated bundle; repeated failure

```

37.I Controlled Execution

```
open read-only transaction
set statement timeout / result limit
execute query
capture result metadata + latency
rollback/close transaction
OUTPUT: result set / aggregate + execution status
FAILURE CONDITIONS: DBMS permission error, timeout, runtime error -> no unsafe retry; return diagnostic
```

38. Technology Stack

Technology	Exact module/purpose	Why selected	Alternative
Python	All orchestration, data models, EEVA, experiments	Fast implementation; strong NLP/DB ecosystem; suitable for two students.	Java / TypeScript backend, but adds unnecessary complexity.
PostgreSQL	Primary Olist DB, authoritative schema/policy source, controlled execution	Strong metadata catalogs, roles, transactions, EXPLAIN and read-only controls.	MySQL possible, but Prompt 2 authority rules are easiest to reproduce in PostgreSQL.
Sentence Transformers	Embeddings for metadata/glossary/docs/query retrieval	Local, reproducible semantic retrieval and compatible with vector stores.	Hosted embeddings; freeze one model if used.
pgvector / simple local index	Semantic evidence index when retrieval needs it	Keeps retrieval local and simple; not a research contribution.	ChromaDB optional; FAISS or in-memory index.
psycopg / SQLAlchemy	Controlled PostgreSQL access and generic database adapters	Supports database_id-based access while keeping one codebase.	Direct psycopg only for the simplest prototype.
NetworkX	Optional schema graph and join-path search	Convenient graph/path algorithms; implementation-only choice.	Custom adjacency graph or SQL catalog recursion.
One frozen instruction-following LLM	Text-to-SQL generation; optional QIM semantic extraction and correction	Keeps research variable focused on EEVA; no LCHR in v1.	A local SQL-capable model if privacy/cost requires it.
SQL parser/validator (e.g., SQLGlot) + PostgreSQL EXPLAIN	Stage 2 parsing/AST checks plus DB-aware preflight	Separates static checks from DB execution; established supporting function.	Another SQL parser; DB-native PREPARE/EXPLAIN checks.
Streamlit; FastAPI SHOULD HAVE	Final-year demo UI; typed service API when time permits	Transparent demo first; API remains engineering support.	Streamlit-only mandatory interface.
Pydantic + JSON/CSV	Typed evidence, obligation, adapter and experiment contracts	Runtime validation plus inspectable/versioned artifacts.	Dataclasses for the earliest prototype.

Technologies intentionally omitted

No mandatory LangChain, multi-agent framework, fine-tuning stack, local-cloud router, or separate hallucination firewall is required for the core experiment. Add them only if implementation needs them, not to make the project look more advanced.

39. Baseline Models

Baseline	Source	Method	What it tests
B0 Direct LLM Text-to-SQL	General generation literature [P1,P20]	q + broad schema -> LLM -> SQL	Reference for direct prompting; expected most exposed to corrupted context.
B1 RAG Text-to-SQL	RAGSQL [P19], Baek et al. [P14]	Retrieve top-k schema/knowledge -> LLM	Separates retrieval quality from evidence validation.
B2 Schema-aware retrieval	FinSQL [P2], LitE-SQL [P18], XiYan-	Schema filter/retriever before generation	Tests whether schema filtering alone
B5 DriftSQL WITHOUT EEVA	Primary ablation	Same initial EvidenceSnapshot -> BaselineGenerationBundle -> same join + LLM + Stage 2	Primary causal baseline; no EEVA trust/recovery/G-C-R logic.
B6 DriftSQL WITH EEVA	Proposed system	Same initial EvidenceSnapshot -> EEVA -> ValidatedEvidenceBundle on GENERATE	Proposed system; CLARIFY may create recovery snapshots.

Primary comparison - frozen invariant

B5 and B6 start from the SAME INITIAL EvidenceSnapshot. All downstream model/prompt, join implementation, Stage-2 rules, correction bound, database and execution settings are identical. New snapshots produced only by B6 through EEVA-approved CLARIFY recovery are treatment effects, not configuration mismatches. Join reasoning must not re-implement EEVA checks.

40. Experimental Design

Exp	Objective	Setup	Baselines	Expected behavior	Metrics	RQ	What it proves
E1 Normal Evidence	Establish clean-data utility and false-rejection cost.	Held-out clean queries + canonical evidence.	B0,B1,B5,B6	B6 should mostly GENERATE and preserve downstream accuracy.	EX, Valid SQL, FRR, latency	RQ1,RQ5	EEVA does not over-block good evidence.
E2 Missing Evidence	Test completeness and recoverability.	Remove mandatory evidence from retrieval; define recoverable vs critical variants.	B1,B3,B5,B6	B6 CLARIFY if recoverable; REJECT if confirmed unavailable.	3-class F1, FAR, recovery rate, EX after recovery	RQ1,RQ2	Missing evidence is detected before generation.
E3 Schema Drift	Test stale/changed schema handling.	Renames/structural drift + stale index with live refresh.	B2,B5,B6	B6 CLARIFY -> refresh -> GENERATE if resolved.	FAR, drift detection, recovery EX, latency	RQ1,RQ3	Robustness to structural evidence drift.
E4 Metadata Inconsistency	Test metadata validity/freshness.	Cryptic column + missing/stale/contradictory description/unit.	B1,B5,B6	B6 should not silently guess when semantics remain unresolved.	Metadata-state accuracy, FAR, EX	RQ2,RQ3	Value beyond schema-only validation.
E5 Business Definition Ambiguity	Test business semantics.	Two plausible glossary definitions for required metric.	B3,B5,B6	B6 -> focused user CLARIFY.	Clarify recall, FAR, post-clarification EX	RQ2,RQ4	Business-semantic component works.
E6 Documentation Conflict	Test cross-source consistency.	Contradictory approved docs / glossary-doc mismatch.	B1,B5,B6	B6 resolves by authority/version or rejects unresolved conflict.	Conflict P/R/F1, FAR, reason accuracy	RQ4	Direct test of cross-source contribution.
E7 Permission Violation	Test adopted authorization hard gate.	Role denies required table/column/operation.	B4,B5,B6	B6 REJECT before generation; DBMS still enforces.	Permission detection, false permit, unsafe generation	RQ1,RQ2	Correct integration of existing access control.
E8 Multiple Evidence Conflicts	Stress non-compensatory rule.	Combine conflict + missing/stale + optional good evidence.	B5,B6	Hard failures dominate; optional evidence cannot mask them.	FAR, macro-F1, reason accuracy	RQ2,RQ4	Integrated state logic.
E9 EEVA Ablation	Isolate causal value.	Same scenarios, identical components; B5 vs B6.	B5,B6	B6 should reduce FAR/unsafe generation with measured FRR/latency cost.	FAR, FRR, EX, error rate, latency	RQ1,RQ5	Primary evidence for contribution.
E10 End-to-End Comparison	Measure system-level utility.	Mixed clean + corrupted held-out cases.	B0-B6	B6 should improve reliability-weighted behavior; raw generation coverage may be lower.	EX on answerable, valid SQL, unsafe generation, correction load	RQ1-RQ5	Full impact/trade-off.

41. Ablation Study

Ablation	Change	What it tests
A. Full DriftSQL + EEVA	All validated EEVA dimensions active.	Reference proposed system.
B. DriftSQL without EEVA	Remove Stage 1 only; all other components identical.	Primary causal effect of EEVA.
C. EEVA Schema-Only	Validate only schema obligations.	Tests whether broader heterogeneous evidence adds value beyond narrow pre-generation schema reliability.
D. EEVA - Conflict Check	Disable cross-source conflict detection.	Measures contribution of semantic/document conflict logic.
E. EEVA - Authorization Gate	Remove Stage 1 permission check but keep Stage 2/DBMS enforcement.	Measures early-rejection benefit; not novelty of access control.
F. EEVA - Recoverability	Collapse CLARIFY into REJECT.	Quantifies utility gained by bounded recovery compared with pure abstention/refusal.

Run internal dimension ablations on a focused 40-60 case stress subset to keep the experiment matrix feasible. B5 vs B6 must run on the full held-out evaluation set.

42. Evaluation Metrics

42.1 EEVA primary metrics

Metric	Role	Why necessary
3-class Macro-F1	Primary classification metric	Treats GENERATE, CLARIFY and REJECT equally despite class imbalance.
Per-class Precision/Recall/F1	Diagnostic	Clarify and Reject errors have different consequences.
False Acceptance Rate (FAR)	Primary reliability metric	Unreliable evidence predicted GENERATE / all unreliable evidence; directly measures EEVA target failure.
False Rejection Rate (FRR)	Primary utility metric	Reliable evidence predicted CLARIFY/REJECT / all reliable evidence; prevents trivial over-rejection.
Evidence Conflict Detection F1	Contribution-specific	Measures cross-source conflict detection independent of final decision.
Permission Violation Detection / False Permit	Safety diagnostic	Required for permission scenario; permission itself is adopted, not novel.
Evidence Coverage Accuracy / Error	Evidence-model diagnostic	Compare predicted mandatory coverage C_m against gold obligation coverage.
Clarification Recovery Rate	CLARIFY utility	Fraction of recoverable cases that become correct GENERATE after bounded recovery.
Reason-code Accuracy	Explainability diagnostic	Checks whether reported missing/conflict/permission reason matches oracle failure type.

42.2 Text-to-SQL metrics

Metric	Priority	Justification
Execution Accuracy (EX)	PRIMARY	Semantic equivalence is better captured by execution than literal string match.
Valid SQL Rate	PRIMARY diagnostic	Measures parse/schema executability before result correctness.
Schema Linking Accuracy / P-R-F1	OPTIONAL diagnostic	Use only if gold schema items are annotated; separates linking from EEVA failure.

Table 42.2 continues on the next page.

Metric	Priority	Justification
Exact Match	SECONDARY	Report if canonical SQL exists; do not drive conclusions because equivalent SQL forms differ.
Execution Error Rate	PRIMARY	Tracks runtime/invalid failures after generation and correction.
SQL Correction Success Rate	SUPPORTING	Evaluates Stage 2 correction separately from EEVA.

42.3 QIM / obligation metrics

Metric	Layer / role	Purpose
Mandatory Obligation Recall	QIM - PRIMARY	Gold mandatory obligations correctly identified / total gold mandatory obligations.
Obligation Precision	QIM diagnostic	Correct predicted canonical obligations / all predicted obligations.
Obligation F1	QIM diagnostic	Balanced quality of predicted R(q) against Gold R(q).
Unsupported Intent Accuracy	QIM diagnostic	Checks early read-only scope precheck separately from EEVA metrics.

42.4 Retrieval/system metrics

Metric	Layer	Purpose
Recall@k / NDCG@k	Retrieval	Was required evidence retrieved and ranked well?
p50/p95 Query Latency	Feasibility	Cost of EEVA, recovery, correction and execution.
Clarification / correction counts	Feasibility	Checks bounded operation (maximum two each).

43. Expected Results

Important: These are directional hypotheses for planning, not experimental findings. Do not copy them into the IEEE Results section as achieved outcomes.

Condition	Expected direction
Clean evidence	B6 should have a low FRR and execution accuracy close to B5; otherwise EEVA is over-conservative.
Missing/stale evidence	B6 should reduce false generation by routing recoverable cases to CLARIFY and critical cases to REJECT.
Business/document conflict	Cross-source checks should reduce FAR relative to RAG/no-EEVA baselines that receive relevant but contradictory context.
Permission violations	Stage 1 should reject denied queries early; Stage 2/DBMS should remain the final enforcement backstop.
Schema drift	Refresh after CLARIFY should recover a meaningful fraction of drift cases and restore downstream EX where a valid mapping still exists.
EEVA ablation	B6 should outperform B5 on FAR/unsafe generation under corrupted evidence, at the cost of some latency and possibly higher FRR.
Self-correction load	If EEVA blocks bad prerequisites early, fewer hopeless SQL candidates should enter the correction loop; measure rather than assume.

44. Limitations

- The novelty claim is scoped to reviewed Text-to-SQL literature; enterprise data-governance/catalog/semantic-layer work may overlap and should be searched before publication.
- Controlled evidence corruptions may be less diverse than real organizational drift; each perturbation must be reviewed for plausibility.
- Gold EEVA decisions and required-evidence obligations require human annotation and may contain subjectivity.
- Business glossary/document authority rules are project policies, not universal enterprise standards.
- LLM output varies by model/version; results may not generalize to every generator.
- EEVA cannot replace DBMS permissions or full security controls.
- Hard rules improve interpretability but may miss nuanced conflicts; overly aggressive rules can increase FRR.
- Exact SQL semantic correctness is difficult to validate automatically; execution accuracy is necessary but not sufficient for every business semantic edge case.
- Prompt 1 selected paper P24 (SV2-SQL) remains technically unverifiable in this corpus.
- Olist is the primary real-world domain; a small LiveSQLBench Base-Lite subset improves external-validity evidence but remains secondary and does not justify broad cross-domain claims.

45. Six-Month Timeline

Month	Main work	Exit criteria
Month 1	Verify Olist source/license/files; profile rows, columns, types, nulls and duplicates; load Olist into PostgreSQL; validate real keys/relationships; automatic schema extraction/hash; register database_id; define generic adapter; begin metadata/glossary/doc/policy evidence.	Olist audit and repeatable load; verified schema snapshot/hash; database_id + adapter skeleton; initial evidence artifacts.
Month 2	Finalize evidence repository; QIM and R(q); direct/semantic retrieval and normalization; alias/semantic mappings; initial question set.	Query -> R(q) -> normalized evidence works; 20 base questions and ~100 evidence variants available for demo/review.
Month 3	Implement EEVA states, hard constraints, conflict detection, source authority, mandatory coverage, reason codes, clarification/recovery loop and controlled evidence scenarios.	Rule tests cover G/C/R, no-progress and RECOVERY_EXHAUSTED; controlled evidence scenarios reviewed.
Month 4	Explainable join reasoning; one frozen LLM Text-to-SQL; SQL Validation Stage 2; at most two correction attempts; controlled execution; Streamlit end-to-end demo.	Unseen Olist question completes the two-stage pipeline; read-only security and bounded correction verified.
Month 5	B0-B6 as feasible; B5/B6 primary experiment; held-out Olist evaluation; ablations; preliminary measured metrics; freeze EEVA rules/prompts before secondary evaluation.	Reproducible Olist experiment runner; B5/B6 differ only by Stage-1 EEVA; preliminary actual metrics recorded.
Month 6	Final Olist evaluation; small LiveSQLBench Base-Lite secondary evaluation; error analysis; final plots/tables; paper revision; demo; documentation and viva preparation.	Olist and Base-Lite results reported separately; final measured metrics, paper/demo and reproducibility package complete.

46. Team Member Responsibilities

Owner	Primary implementation responsibilities	Cross-review
Member 1	PostgreSQL/schema/migrations; evidence repository + metadata/glossary; EEVA hard rules/decision engine; SQL validation/SFO/executor.	Member 2 reviews data semantics and generated cases; both review labels/errors.
Member 2	QIM/AASE; retrieval/MAL/SDARE; conflict detection + clarification; join reasoning + LLM gateway; experiments/graphs/UI.	Member 1 reviews R(q) schema needs, authoritative sources and SQL safety interfaces.
Shared	Paper/report/demo/viva; B5/B6 experiment ownership; evidence annotation and cross-boundary integration.	Both members must explain the complete two-stage system.

This section now matches the Production Blueprint implementation ownership matrix; Appendix E remains the freeze summary.

47. Risks and Mitigation

Risk class	Risk	Mitigation
Easy	PostgreSQL setup, basic Streamlit UI, direct schema extraction.	Finish early; do not spend research time polishing UI.
Moderate	RAG retrieval, QIM structured extraction, NetworkX join graph, SQL parsing/validation.	Use simple documented interfaces and unit tests; avoid unnecessary agents.
Difficult	Gold evidence obligations, cross-source conflict logic, recoverability semantics, fair B5/B6 isolation.	Freeze annotation guide; independent review; scenario oracles; automated reproducibility tests.
Highest research risk	EEVA may not improve reliability enough, or may simply reproduce existing abstention behavior.	Make heterogeneous evidence and cross-source conflict scenarios central; report negative/neutral results honestly.
LLM risk	Version drift, hallucinated SQL, nondeterminism, API cost.	Freeze one model/config; cache experiment inputs/outputs where allowed; same model across baselines.
Dataset risk	Olist source/license/files or exact schema/relationships may remain unverified.	Audit actual Olist materials in Month 1; block unverified claims and affected tasks rather than inventing or substituting unrelated data.
Evaluation risk	Synthetic corruption may be unrealistic; gold labels may be circular.	Document perturbation rationale; faculty/domain review subset; keep dev/test split.
Publication risk	Global novelty may overlap enterprise governance literature.	Use conservative language; add broader novelty search before submission; position contribution as integrated Text-to-SQL evidence-state validation.
Latency risk	Clarification/probes/correction increase runtime.	Bound EEVA clarifications and SQL corrections to two cycles; report p50/p95.

Simplification path

If schedule slips, preserve: QIM -> evidence retrieval -> EEVA -> Generate/Clarify/Reject -> simple join reasoning -> one LLM -> SQL validation -> read-only execution. Remove optional ChromaDB (use small in-memory retrieval), NetworkX (use adjacency lists), Streamlit polish, extra baselines B2-B4, and internal dimension ablations before touching the central B5 vs B6 experiment.

48. Future Work

- Expand the small LiveSQLBench Base-Lite secondary study to broader multi-database benchmarks only after the Olist result is established.
- Explore Random Forest or XGBoost only as stretch soft-decision support after the constrained rule baseline; learned scores must never override hard constraints.
- Integrate enterprise data catalogs/semantic layers and formal provenance standards after broader literature review.
- Add temporal monitoring that automatically detects schema/metadata/document drift and refreshes evidence indexes.
- Explore privacy-preserving or local LLM deployments; LCHR can be revisited as an engineering optimization, not the core contribution.
- Use multi-turn clarification state when users refine business definitions over several turns.
- Investigate learned conflict resolution or source reliability models with human-in-the-loop governance.
- Benchmark against broader policy-aware and pre-synthesis verification systems once implementations/datasets are available.

49. Research Paper Contribution

49.1 Recommended paper framing

Paper thesis

We investigate whether query-conditioned validation of heterogeneous enterprise evidence before SQL generation reduces false and unsafe Text-to-SQL generation under incomplete, stale, conflicting and unauthorized context.

49.2 Claims the paper may make if experiments support them

- EEVA provides a formal evidence-state representation and non-compensatory decision policy over multiple enterprise evidence types.
- The B5-vs-B6 ablation shows the incremental effect of pre-generation evidence validation while holding retrieval/generation/Stage 2 constant.
- Controlled evidence-failure scenarios make false acceptance, false rejection, conflict detection and clarification recovery measurable.
- Two-stage validation separates prerequisite evidence reliability from generated-SQL correctness.

49.3 Claims the paper must not make

- "The first safe Text-to-SQL system."
- "The first pre-generation validation method."
- "The first system to clarify or reject."
- "The first permission-aware Text-to-SQL system."
- "RAG/SQL validation/self-correction are novel contributions of DriftSQL."
- Any performance number before the experiment is actually run.

49.4 Suggested IEEE paper structure

Paper section	Content
I. Introduction	Enterprise context unreliability problem; contributions; conservative positioning.
II. Related Work	Schema/retrieval; ambiguity/abstention; evidence reasoning; enterprise

Paper section	Content
IV. DriftSQL and EEVA	Architecture, source authority, hard/soft constraints, decision policy, clarification loop.
V. Experimental Setup	Olist primary evidence layer and held-out cases; controlled evidence variants: baselines/metrics; separate LiveSQLBench Base-Lite secondary evaluation after freeze; no combined score.
VI. Results	B5/B6 primary results, scenario breakdown, ablations, latency, error analysis.
VII. Discussion / Threats	Novelty scope, synthetic corruption, single domain, DBMS security boundary.
VIII. Conclusion	What the evidence shows; no overclaiming.

50. Final Viva Explanation

50.1 30-second answer

"DriftSQL is an enterprise Text-to-SQL system. The main problem we study is that the LLM can receive incomplete, stale, conflicting, or unauthorized enterprise context and still generate plausible SQL. Our core component, EEVA, validates the evidence needed for the specific user query before SQL generation and returns GENERATE, CLARIFY, or REJECT. If it generates, the SQL then goes through a separate validation and bounded correction stage before read-only execution. Our main experiment compares the exact same DriftSQL pipeline with and without EEVA to measure whether pre-generation evidence validation reduces false and unsafe SQL."

50.2 Key differentiation question

If asked: "Isn't this just SQL validation?"

No. EEVA runs before SQL exists and asks whether the evidence supplied to the generator is trustworthy enough. SQL Validation runs after SQL exists and asks whether that candidate query is correct, safe and executable. They solve different failure stages and are evaluated separately.

50.3 If asked: "What is actually new?"

"DriftSQL proposes and investigates a query-conditioned enterprise evidence-validation mechanism for Text-to-SQL: Required Evidence Obligations $R(q)$, heterogeneous evidence, source authority/version/provenance, non-compensatory hard constraints, recoverability, and GENERATE/CLARIFY/REJECT before SQL generation. Retrieval, clarification, permission checking, database probing, SQL validation, self-correction and join reasoning are supporting techniques, not separate novelty claims."

50.4 If asked: "How will you prove EEVA matters?"

"The primary ablation is DriftSQL without EEVA versus DriftSQL with EEVA, using the same query understanding, retrieval, LLM, join logic, SQL validation and correction. We deliberately create controlled evidence failures and measure false acceptance, false rejection, Generate/Clarify/Reject macro-F1, conflict detection, clarification recovery, execution accuracy and latency."

Appendix A. 30-Paper Traceability

The original project materials refer to a 30-paper collection but Prompt 1 explicitly selected 25 papers for detailed analysis. The exact membership of the original "30" is not uniquely recoverable from the supplied prompt documents. To avoid inventing membership, this appendix uses the 25 selected papers plus five additional, clearly identifiable full papers from the uploaded corpus (E26-E30). E26-E30 are supplementary traceability only and do not overwrite the Prompt 1 base-paper decision.

No.	Paper	Venue	Year	Area	Main method	Dataset	DriftSQL component	Existing limitation	Adopt	Change	EEVA rel.	Rel.
1	Few-shot Text-to-SQL Translation using Structure and Content Prompt Learning	PACM Data	2023	Few-shot generation	Structure/content prompt learning	Spider; CoSQL; GeoQuery	Generation	Prompt/init sensitivity	Decomposition/constrained decoding	Add EEVA before generation	Low	Medium
2	FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis	SIGMOD Companion	2024	Enterprise schema retrieval	Cross-Encoder + PEFT + calibration	BULL	QIM/retrieval/AASE	Domain-specific; no integrated evidence gate	Wide-schema retrieval/terminology handling	Broaden to multi-source evidence + EEVA	Partial	High
3	CodeS: Towards Building Open-source Language Models for Text-to-SQL	PACM Data	2024	Open Text-to-SQL models	SQL pretraining + SFT + schema linking	Spider; BIRD + robustness	Generator/local deployment	Model-centric	Private/open model motivation	Treat generator as replaceable after EEVA	Low-partial	Medium
4	Reliable Text-to-SQL with Adaptive Abstention	PACM Data	2025	Reliability/abstention	BPP + conformal prediction	BIRD; Spider	EEVA-adjacent pre-gate	Schema/model uncertainty only	Selective pre-generation control	Generalize validation object to evidence state	Strong partial	High
5	Automated Validating and Fixing of Text-to-SQL Translation with Execution Consistency	PACM Data	2025	SQL validation	Execution consistency + equivalence	Spider; BIRD	Stage 2 validation	After generation; cost/scaling	Execution validation/repair	Keep separate from EEVA	Complementary	High
6	OpenSearch-SQL: Enhancing Text-to-SQL with Dynamic Few-shot and Consistency Alignment	PACM Data	2025	Agentic generation	Dynamic few-shot + alignment + refinement	BIRD	RAG/self-correction	No multi-source evidence gate	Dynamic retrieval/refinement	Insert EEVA before generation	Partial	High
7	DIVER: A Robust Text-to-SQL System with Dynamic Interactive Value Linking and Evidence Reasoning	PACM Data	2026	Evidence reasoning	Interactive value linking + CoTF	BIRD; Spider; Dr.Spider; Spider-DK	Retrieval/CARD	Constructs evidence; does not certify all sources	Dynamic evidence discovery/probes	Use as CLARIFY acquisition, then EEVA	Strong partial	High
8	DeepEye-SQL: A Software-Engineering-Inspired Text-to-SQL Framework	PACM Data	2026	System reliability	Robust linking + N-version + checkers	BIRD; Spider; Spider 2.0	Join/Stage 2	Operationally complex; schema-link residuals	Checker pipeline/relational closure	Add pre-generation evidence gate	Partial	High
9	Reward-SQL: Boosting Text-to-SQL via Stepwise Execution-Aware Reasoning and Process-Supervised Rewards	PACM Data	2026	Reasoning/RL	CoCTE + process rewards	BIRD; Spider	Generation/reasoning	Training complexity; not evidence governance	Executable stepwise reasoning	Do not substitute for EEVA	Low	Medium
10	Benchmarking and Improving Text-to-SQL Generation under Ambiguity	EMNLP	2023	Ambiguity	AmbiQT + LogicalBeam	AmbiQT/Spider-derived	QIM/CLARIFY	Synthetic/English ambiguity	Ambiguity taxonomy	Route ambiguity to evidence/user clarification	Partial	High
11	Know What I Don't Know: Handling Ambiguous and Unanswerable Questions for Text-to-SQL	Findings ACL	2023	Answerability	DTE + counterfactual data	User study + generated sets	QIM/decision explanations	Some classes uncovered	Detect/explain when not to answer	Broaden trigger to enterprise evidence state	Strong partial	High
12	Re-appraising the Schema Linking for Text-to-SQL	Findings ACL	2023	Schema linking	EMSL re-appraisal + grammar linking	Spider variants	AASE/schema robustness	Older model-scale constraints	Avoid brittle exact lexical linking	Use semantic metadata + EEVA obligations	Low-partial	Medium
13	Exploring Schema Generalizability of Text-to-SQL	Findings ACL	2023	Schema generalization	Schema transformations/test suites	Spider/transformed suites	SDARE motivation	Synthetic transformations	Controlled schema-drift idea	Use live refresh + evidence-state validation	Partial	High
14	Knowledge Base Construction for Knowledge-Augmented Text-to-SQL	Findings ACL	2025	Knowledge augmentation	KB construction + vector retrieval	BIRD; Spider; CSTINSIGHT	RAG/MAL/CARD	Coverage + privacy/access risks	Business/context knowledge retrieval	Validate retrieved knowledge before use	Partial	High
15	Optimizing Reasoning for	Findings ACL	2025	Execution-guided training	ExCoT-DPO	BIRD; Spider	Generation/correction	Reasoning	Execution feedback	Keep as optional Stage	Low	Medium

No.	Paper	Venue	Year	Area	Main method	Dataset	DriftSQL component	Existing limitation	Adopt	Change	EEVA rel.	Rel.
	Text-to-SQL with Execution Feedback							faithfulness/domain limits		2/training support		
16	SDE-SQL: Enhancing Text-to-SQL Generation via Self-Driven Exploration with SQL Probes	ACL	2026	DB exploration	Self-driven SQL probes	BIRD; Spider	Clarify retrieval/CARD	Prompt dependence/overhead	Safe evidence acquisition	Subject probe facts to EEVA	Partial	High
17	PV-SQL: Synergizing Database Probing and Rule-based Verification for Text-to-SQL Agents	Findings ACL	2026	Probing/verification	DB probes + executable rules	BIRD; Spider/Mini-Dev	Retrieval + Stage 2	Rule set incomplete	Grounding + rule verification	Split Probe upstream; Verify Stage 2	Strong partial	High
18	LIE-SQL: Lightweight and Efficient Text-to-SQL with Vector Schema Linking and Self-Correction	Findings EACL	2026	Efficient retrieval/correction	Vector retriever + execution correction	BIRD; Spider 1.0	Vector retrieval/Stage 2	Fixed-k false positives; semantic errors	Lightweight retrieval/correction	Validate richer evidence pre-generation	Partial	High
19	RAGSQL: Context Retrieval Evaluation on Augmenting Text-to-SQL Prompts	IEEE IS	2024	RAG retrieval	Dense embedding retrieval evaluation	BIRD-derived corpus	RAG/CARD	Retrieval relevance != trust	Recall/NDCG methodology	Use RAG then EEVA	Partial	High
20	Leveraging Large Language Model for Enhanced Text-to-SQL Parsing	IEEE Access	2025	Parsing/generation	Prompt search + embeddings + syntax decoder	Spider	QIM/generation	No evidence-validation focus	Syntax-aware generation	Place after EEVA	Low	Medium
21	XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL	IEEE TKDE	2026	Multi-generator/schema coverage	Multi-path schema filters + generators + selector	BIRD; Spider	Schema coverage/generation	Multi-model complexity	Alternative schema candidates	Validate candidate evidence state before generation	Partial	High
22	DB-GPT: Large Language Model Meets Database	Data Science and Engineering	2024	LLM+DB systems	DB prompts/fine-tuning/model vision	TPC workloads; IMDB	Enterprise context/MAL	Vision paper; no unified T2SQL metric	DB-specific context/privacy motivation	Narrow to concrete evidence gate	Low	Medium
23	LLM-Based Text-to-SQL for Real-World Databases	Springer accepted manuscript	2025	Real-world databases	Benchmark + LLM-friendly views/samples	IndDB; Mondial	Enterprise evidence/MAL	IndDB proprietary; model dependence	Context engineering + samples	Validate agreement/authorization before generation	Partial	High
24	SV2-SQL: BERT Slot Filling, Value Extraction, and Verification	Full paper missing	2024*	Unverifiable	Not verifiable	Not verifiable	Unmapped	Full source absent	None until source available	No evidence-based change claim	Unverifiable	Low
25	ARE-SQL for Improving Semantic Alignment in Chinese Multi-round Text-to-SQL Parsing	Discover Computing	2026	Conversational alignment	State tracking + relation-aware attention	CHASE	QIM/clarification state	One Chinese dataset; complexity	Multi-turn alignment	Only if multi-turn clarification retained	Low	Low-Medium
26	Enhancing Text-to-SQL Capabilities of Large Language Models through Tailored Promptings	LREC-COLING	2024	Prompting/schema linking	SL, CC, SL+CC, RS prompts	Spider	Generation/schema linking	Prompt-focused; no enterprise evidence governance	Schema-link prompt/decomposition ideas	Keep generation after EEVA	Low	Medium
27	A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going?	IEEE TKDE	2025	Survey	Lifecycle taxonomy of LLM Text-to-SQL	N/A - survey	Literature/background	Survey does not implement EEVA	Taxonomy/challenges	Use to contextualize, not as base implementation	Background	Medium
28	Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL	IEEE TKDE	2025	Survey	Survey/taxonomy of LLM Text-to-SQL	N/A - survey	Literature/background	Survey, not a concrete evidence validator	Challenges/benchmarks/system taxonomy	Use for broader positioning	Background	Medium
29	Improving the Accuracy of Text-to-SQL Tools based on LLMs for Real-World Relational Databases	DEXA	2024	Real-world RAG	Generate synthetic NL-SQL examples + similarity RAG	Production RW-RDB; Mondial	Enterprise RAG/context	Retrieval/examples do not certify cross-source evidence	Database-specific RAG + documentation use	Add EEVA trust/consistency gate	Partial	High
30	GradeSQL: Outcome Reward Models for Intelligent Text-to-SQL Generation from LLMs	Journal of Intelligent Information Systems	2026	Test-time verification	Outcome Reward Models rank candidate SQL	BIRD; Spider	Stage 2 verification	Candidate-SQL verification, not prerequisite evidence governance	Probabilistic SQL verifier concept	Keep as possible Stage 2 baseline/future work	Complementary	Medium-High

Appendix B. DriftSQL Module Traceability

Component	Purpose	Class	Related papers	Existing technique?	DriftSQL adaptation	Research contribution?	Evaluation
EEVA	Validate heterogeneous enterprise evidence before generation.	CORE	P4,P7,P11,P14,P17,P19,P23 + Prompt2 cross-checks	No exact integrated equivalent verified.	Query-conditioned obligations + cross-source hard/recoverable states.	YES - central proposed contribution.	Macro-F1, FAR/FRR, conflict/recovery metrics, B5 vs B6.
QIM	Intent/entity/ambiguity extraction + R(q).	SUPPORTING	P6,P10,P11,P25	Underlying techniques established.	Add explicit Required Evidence Obligations output.	No.	Obligation accuracy (optional), downstream error analysis.
LCHR	Route local/cloud models.	OPTIONAL	P3,P18,P22	Motivation partial; exact router not needed.	Omit from v1 research architecture.	No.	None unless implemented.
AASE	Handle abbreviations/naming mismatch.	OPTIONAL / MERGE	P2,P12,P23	Problem established.	Aliases/synonyms/descriptions inside QIM/retrieval.	No.	Schema-link diagnostics if annotated.
SDARE	Refresh embeddings/evidence under schema drift.	OPTIONAL / MERGE	P13,P16,P23	Drift motivation established.	Refresh schema/metadata index in CLARIFY loop.	No.	E3 drift recovery, latency.
MAL	Normalize metadata/context evidence.	SUPPORTING	P14,P22,P23	Underlying abstraction established.	Typed EvidenceItem + provenance/version/authority.	Supports C1 but not standalone novelty.	Evidence validity/coverage diagnostics.
SFO	Observe semantic/execution failures.	SUPPORTING / MERGE	P5,P6,P8,P17,P18	Established.	Merge into Stage 2 validator diagnostics.	No.	Execution error/correction success.
HF	Hallucination firewall.	REMOVE	P4,P6,P8,P11 as underlying controls	Exact named module not grounded; duplicates EEVA/checkers.	Remove separate box.	No.	N/A.
CARD	Adaptive retrieval depth.	SUPPORTING / MERGE	P7,P14,P16,P19	Adaptive retrieval broadly established.	Use EEVA CLARIFY recovery plan to expand retrieval.	No.	Recovery rate, cycles, Recall@k.
ChromaDB	Vector evidence index.	SUPPORTING IMPLEMENTATION	P14,P18,P19	Vector retrieval established; product not research novelty.	Optional local vector store.	No.	Retrieval Recall@k/NDCG@k.
NetworkX	Schema graph/join paths.	SUPPORTING IMPLEMENTATION	P8,P17,P23	Graph/join reasoning established.	Implement validated relationship graph.	No.	Join/schema diagnostics, EX.
RAG	Retrieve enterprise evidence.	SUPPORTING	P7,P14,P19	Established.	Retrieval feeds EEVA instead of generator directly.	No.	Recall@k/NDCG@k + EEVA FAR separation.
Explainable Join Reasoning	Choose and explain valid joins.	SUPPORTING	P8,P17,P23	Established in related forms.	Restrict graph to validated evidence and expose path.	No.	Join correctness / downstream EX.
SQL Validation	Validate generated SQL.	SUPPORTING	P5,P8,P17,P18	Established.	Separate Stage 2 with structured diagnostics.	No.	Valid SQL, execution errors.
Controlled Execution	Read-only validation/result execution.	SUPPORTING	P5,P16,P17,P18	Broadly established.	DBMS read-only role, timeouts, result limits.	No.	Execution error, latency.
Self-Correction	Repair invalid candidate SQL.	SUPPORTING	P6,P8,P17,P18	Established.	Bound to 2 attempts and never bypass EEVA.	No.	Correction success rate.
Explainable Result	Expose decision/evidence/join/validation summary.	SUPPORTING	P7,P8,P11,P17	Component-level explanation established.	Unified audit summary without hidden chain-of-thought.	Supports usability, not novelty.	Reason-code accuracy; qualitative demo.

Appendix C. Novelty Traceability and Targeted Cross-Check

Existing paper(s)	Existing method	Limitation	DriftSQL adaptation	EEVA contribution	Experiment
P4 RTS	Calibrated schema-link uncertainty + abstention	Scope is model/schema-link uncertainty	Generalize validation object to enterprise obligations	Non-compensatory evidence-state gate	E3/E9; FAR/FRR
P7 DIVER + P16 SDE-SQL	Interactive evidence acquisition/probes	Evidence constructed but not certified across all sources	Use probes only as CLARIFY recovery and validate returned facts	Recoverability-aware loop	E2/E8; recovery rate
P14 Baek et al. + P19 RAGSQL	Knowledge/context retrieval	Relevant context can be stale/conflicting/unauthorized	Normalize and validate before generation	Retrieval != trust	E4/E6; Recall@k vs FAR
P10/P11 + Prompt2 X5	Ambiguity/unanswerability/clarification	Focus is question ambiguity	Separate query ambiguity from evidence ambiguity	Evidence-grounded clarification reason codes	E5; class-wise F1
Prompt2 X1-X3	Role-aware refusal/policy compliance	Permission is already an established task	Adopt as hard EEVA constraint + DBMS enforcement	Permission is adopted, not novel	E7; false permit
P5/P8/P17/P18	SQL validation and repair	Acts after candidate SQL exists	Keep Stage 2 and compare with/without Stage 1	Two-stage reliability architecture	E10; errors/correction success
P13/P23 + Prompt2 X4	Schema generalization, enterprise context, metadata extraction	Shows context can be stale/poor, not a unified validity gate	Track version/provenance; test stale snapshots	Evidence freshness/drift dimension	E3/E4
Prompt2 X6 VET	Pre-synthesis executable trace verification	Validates reasoning trace vs DB, not multi-source enterprise evidence	Keep EEVA on evidence prerequisites	Evidence governance distinct from trace verification	E8/E9

Prompt 2 targeted cross-check sources

ID	Source	Venue/year	Why it matters
X1	Role-Conditioned Refusals: Evaluating Access Control Reasoning in Large Language Models	Findings EACL 2026	Used to classify permission-aware refusal as existing.
X2	Benchmarking Text-to-SQL under Role-Based Access Control	arXiv 2026	Additional evidence that RBAC allow/deny decisions are established.
X3	Policy-Conditioned Constrained Decoding for Column-Level Access Control in Text-to-SQL	arXiv 2026	Further weakens any permission-validation novelty claim.
X4	Automatic Metadata Extraction for Text-to-SQL	arXiv 2025	Classifies metadata acquisition/enrichment as existing.
X5	PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries	NAACL 2025	Classifies clarification conversations as existing.
X6	VET: Verifiable Execution Tracing for Reliable Text-to-SQL Generation	Findings ACL 2026	Narrows EEVA away from generic pre-synthesis verification.

Novelty scope warning

X1-X6 are targeted checks inherited from Prompt 2, not an exhaustive global novelty search. Before submission, add a broader search of enterprise metadata governance, data catalogs, semantic layers, provenance and policy-aware database interfaces.

Appendix D. Final Consistency Check

Check	Result	Blueprint resolution
Title matches project idea	PASS	Enterprise Query Intelligence + Validation; EEVA remains central.
Problem leads naturally to EEVA	PASS	Problem is unreliable prerequisite evidence before generation.
Literature supports research gap	PASS WITH SCOPE	Supports the scoped query-conditioned enterprise evidence-validation mechanism, not a global first claim.
EEVA distinguishable from existing work	PASS WITH CONDITION	Must remain heterogeneous evidence-state validation; schema confidence or SQL checking alone would collapse into prior work.
No unsupported novelty claims	PASS	Permission, clarification, abstention, RAG, metadata enrichment, SQL validation and self-correction are existing/adopted.
One architecture only	PASS	Single QIM -> R(q) -> retrieval/normalization -> EEVA -> decision -> join -> LLM -> Stage 2 -> bounded correction/execution -> result pipeline; database_id/adapters do not change the architecture.
Methodology matches architecture	PASS	Data objects, algorithms and pseudocode implement the same flow.
Experiments evaluate EEVA	PASS	E2-E8 test dimensions; E9 isolates causal value; E10 measures system effect.
EEVA ablation exists	PASS	B5 vs B6 full test + focused dimension ablations.
Baselines appropriate	PASS	Direct, RAG, schema-aware, clarify/abstain, permission-aware, identical no-EEVA pipeline.
Metrics measure contribution	PASS	Macro-F1, FAR/FRR, conflict/recovery, EX, error/latency.
Feasible for two students	PASS WITH SCOPE CONTROL	Olist primary; tiered 60-80 minimum / ~100 preferred questions; max-two clarification and correction bounds; one frozen LLM; optional modules merged/removed.
Olist primary + Base-Lite secondary	PASS WITH VERIFICATION	One primary Olist study; small LiveSQLBench Base-Lite subset only after freeze, using the same codebase and separate results.
No unnecessary models/technologies	PASS	One frozen LLM; database_id + generic adapters; pgvector/simple retrieval; ChromaDB/NetworkX optional; no LCHR/HF research box.
Literature/gap/novelty/method/experiments describe same system	PASS	All center on query-conditioned multi-source evidence validation before generation.

Final research position

DriftSQL investigates whether a formal, query-conditioned validation of heterogeneous enterprise evidence before SQL generation can reduce false and unsafe Text-to-SQL generation under incomplete, stale, conflicting and unauthorized enterprise context, while preserving acceptable utility and latency.

Source and Evidence Discipline

P1-P25 refer to the selected papers summarized and verified in the Prompt 1 literature foundation. Prompt 2 provides the validated research gap, novelty classification, EEVA formalization, baselines, dataset plan, experiments and targeted novelty cross-check X1-X6. Prompt 3 specifies this Master Blueprint structure and non-negotiable architecture. P24 remains unverifiable because its full paper was absent. E26-E30 are additional identifiable papers from the uploaded corpus included only for the requested 30-paper traceability extension.

IMPLEMENTATION FREEZE: PASS. The six pre-implementation specification issues are resolved in Appendix E, which supersedes ambiguous earlier wording for those topics. Implementation may begin with DATA-001.

Appendix E. Implementation Freeze - Resolved Pre-Implementation Specifications

Authority of this appendix. These rules resolve the six pre-implementation ambiguities identified during final review. For these six topics, this appendix supersedes older incomplete or inconsistent wording. The core DriftSQL architecture and EEVA research position are unchanged.

Issue	Frozen resolution
1. B5 baseline contract	B5 and B6 start from the same INITIAL EvidenceSnapshot . B5 bypasses EEVA and creates a BaselineGenerationBundle . B6 passes the same initial snapshot through EEVA and creates a ValidatedEvidenceBundle only on GENERATE . All downstream model, prompt, join implementation, Stage-2, correction, database and execution settings are identical.
2. R(q) correctness	Create one manually reviewed Gold R(q) per base question for offline evaluation. Gold R(q) is never supplied to runtime. Score canonical obligations and report Mandatory Obligation Recall plus Precision/F1 where practical.
3. Unsupported queries	QIM performs an early read-only scope precheck. Unsupported write/DDL/admin requests return final REJECT with reason UNSUPPORTED_OPERATION at stage QIM_PRECHECK . Generator, correction and executor are not called.
4. Clarification authority	A user may clarify intent, filters or select among approved interpretations, but cannot override authoritative schema, relationships, policy/permissions, provenance, glossary or verified enterprise facts. Clarification always returns through retrieval/normalization and EEVA.
5. Freshness / staleness	Currentness is source-specific: live PostgreSQL catalog for schema/relationships, active DB policy for authorization, current approved/effective versions for glossary/docs, and the current approved metadata snapshot. No arbitrary global time threshold is used.
6. Team ownership	Use the Production Blueprint responsibility matrix as the implementation authority. The research remains jointly owned.

E.1 Decision 1 - Exact B5 versus B6 baseline contract

Shared start: QIM, R(q), direct/semantic retrieval and normalization produce one immutable **initial EvidenceSnapshot** keyed by `request_id` and `database_id`. That exact initial snapshot is persisted for the paired B5/B6 run.

B5 (without EEVA): Initial **EvidenceSnapshot** -> **BaselineGenerationBundle** -> the same join implementation -> same frozen LLM -> Stage-2 SQL Validation -> bounded correction -> controlled execution. The baseline bundle performs only common format validation and mandatory environment-level access filtering shared by both systems. It does not perform EEVA authority, freshness, conflict, recoverability, coverage or G/C/R logic.

B6 (with EEVA): The same initial **EvidenceSnapshot** -> EEVA. **GENERATE** freezes a **ValidatedEvidenceBundle**; **CLARIFY** may create later recovery **EvidenceSnapshots** and re-enter EEVA; **REJECT** stops before generation. Recovery snapshots are treatment outputs and do not invalidate the pair.

Experimental invariant: the paired configuration diff must show that `eeva_enabled` is the only research intervention at the start. The same question, initial **EvidenceSnapshot ID**, `database_id`, `model/version`, `prompt`, `decoding settings`, join implementation, Stage-2 rules, correction bound and execution controls are used.

E.2 Decision 2 - Gold R(q) and QIM evaluation

Create **one Gold R(q) per base question**, not per evidence variant. Gold R(q) is an offline, manually reviewed oracle derived from the verified database schema and approved enterprise evidence. Runtime QIM must generate R(q) itself and must never read Gold R(q).

Canonical scoring rule: after approved alias normalization, compare obligations using **database_id + evidence_type + canonical_target_id + constraint_type**. Do not score free-form wording. Report Mandatory Obligation Recall as the primary QIM metric and obligation Precision/F1 where practical.

E.3 Decision 3 - Unsupported query handling

QIM performs an early scope precheck. **INSERT**, **UPDATE**, **DELETE**, **DROP**, **ALTER**, **CREATE** and other unsupported write/DDL/admin requests return final **REJECT** with reason **UNSUPPORTED_OPERATION** and stage **QIM_PRECHECK**. Generator, correction and executor calls are prohibited; these precheck rejects are excluded from EEVA three-class metrics unless separately analyzed.

Appendix E. Implementation Freeze - Resolved Pre-Implementation Specifications

E.4 Decision 4 - Clarification authority boundary

A user clarification may resolve user intent, filters, requested interpretation, or selection among already approved business meanings. It may not override live schema or relationship facts, DB permissions/policy, provenance/integrity requirements, source authority, or authoritative glossary/document rules.

After a valid clarification: update QueryIntent and affected R(q) obligations -> execute only approved recovery actions -> retrieve/refresh authoritative evidence -> rebuild/update EvidenceSnapshot -> run EEVA again. Direct clarification -> SQL generation is prohibited. The maximum-two-cycle and no-progress rules remain unchanged.

E.5 Decision 5 - Source-specific freshness rules

Freshness is defined by agreement with the authoritative current state for each evidence category, not by one arbitrary age threshold. Freeze these rules before held-out evaluation.

Evidence type	Authoritative current state	When stale/recoverable
Schema / relationships	Active PostgreSQL catalog / information_schema + approved current relationship snapshot.	Cached snapshot/hash differs while live refresh is available.
Authorization	Active PostgreSQL role/policy configuration.	Cached policy differs; permission denial itself remains a hard failure.
Business glossary	Latest approved definition whose effective period applies to the query.	Older/superseded approved definition is retrieved while a current one exists.
Documentation	Latest applicable approved document version by authority and effective date.	Older applicable version is retrieved and can be replaced by the current approved source.
Metadata	Current approved metadata version/snapshot linked to the active schema.	Version/hash no longer matches the approved/current state and refresh is available.

E.6 Decision 6 - One implementation ownership matrix

The Production Implementation Blueprint responsibility matrix is the implementation authority. This table establishes primary implementation ownership only; research authorship and B5/B6 results remain shared.

Area	Primary owner	Shared review
PostgreSQL, schema, migrations	Member 1	Member 2 reviews data semantics
Evidence repository, metadata/glossary	Member 1	Both approve contracts and definitions
QIM and AASE	Member 2	Member 1 reviews R(q) schema needs
Retrieval, MAL, SDARE	Member 2 primary	Member 1 reviews authoritative sources
EEVA hard rules and decision engine	Member 1	Both review labels and errors
Conflict detection and clarification	Member 2	Both review recovery behavior
Join reasoning and LLM gateway	Member 2	Member 1 reviews SQL safety interfaces
SQL validation, SFO and executor	Member 1	Member 2 reviews generated cases
Experiments, graphs and UI	Member 2 primary	Both own B5/B6 and results
Paper, report, demo and viva	Shared	Both must explain the complete system

Appendix E. Implementation Freeze - Start-of-Implementation Contract

E.7 Definition of ready for coding

The architecture is frozen for the first implementation cycle. No further redesign is required before coding. Start with DATA-001: verify the actual Olist source, license, files, schema, keys and relationships; then create the database_id adapter and typed contracts.

The six resolved decisions must be represented in code, tests and experiments. They do not change the core architecture: QIM -> R(q) -> retrieval/normalization -> EEVA -> Generate/Clarify/Reject -> join reasoning -> one frozen LLM -> Stage-2 SQL Validation -> bounded correction -> controlled read-only execution -> explainable result.

E.8 Minimum implementation acceptance checks

Check	Acceptance
B5/B6 isolation	Paired runs use the same request, question and INITIAL EvidenceSnapshot ID; only eeve_enabled differs initially. B6 recovery snapshot IDs are logged as treatment outputs.
Gold R(q)	Every frozen base question has one reviewed Gold R(q); runtime code cannot access gold annotations; QIM metrics are computed offline with canonical obligation keys.
Unsupported operations	Unsupported/write requests stop at QIM_PRECHECK; generator_called=false and execution_called=false.
Clarification authority	User clarification may change intent/selection but cannot change authoritative permission/schema/policy/provenance facts; every accepted clarification re-enters EEVA.
Freshness	Schema/policy/version fixtures produce deterministic current/stale states using the frozen source-specific rules.
Ownership	The task board follows the Production responsibility matrix and both members review cross-boundary interfaces.

E.9 Codex authority rule

For the six frozen topics, Codex must follow Appendix E exactly if older wording is ambiguous. For all other topics, the Master Blueprint controls research meaning and the Production Blueprint controls engineering/task order. Never invent Olist schema facts before DATA-001 verifies them.

IMPLEMENTATION STATUS: READY TO START. Dataset facts remain subject to DATA-001 verification; no Olist table, column, key or relationship is assumed until verified from the actual source.